

IMPLEMENTATION OF DATA HIDING METHODS FOR RESOURCE CONSTRAINED IoT DEVICES

A PROJECT REPORT

Submitted by

NISHMANUL HAQ S (Reg. No. 9517202203103)
SHAH MUHAMMED ILYAS M (Reg. No. 9517202203141)

in partial fulfillment for the award of the degree

of

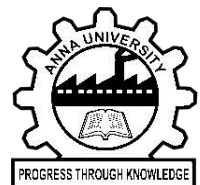
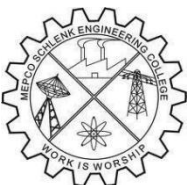
BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI
(An Autonomous Institution affiliated to Anna University Chennai)



April 2026

BONAFIDE CERTIFICATE

Certified that this project report titled “**IMPLEMENTATION OF DATA HIDING METHODS FOR RESOURCE CONSTRAINED IoT DEVICES**” is the bonafide work of **NISHMANUL HAQ S (Reg. No: 9517202203103)** and **SHAH MUHAMMED ILYAS M (Reg. No: 9517202203141)** who carried out the research under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. N. Sivasankari M.E., Ph.D.,

**Internal guide
Associate Professor
Department of Electronics and
Communication Engineering,
Mepco Schlenk Engineering College,
Sivakasi- 626005**

Dr. R. Shantha Selvakumari M.S., Ph.D.,

**Head of the Department
Senior Professor and Head
Department of Electronics and
Communication Engineering,
Mepco Schlenk Engineering College,
Sivakasi-626005**

Submitted for viva-Voce Examination held at **MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI (AUTONOMOUS)** on

Internal Examiner

External Examiner

ABSTRACT

The rapid proliferation of Internet of Things (IoT) devices across industrial, domestic, and commercial environments has brought data security to the forefront of embedded systems research. Conventional encryption algorithms such as AES and RSA, while cryptographically robust, impose computational and memory overheads that are incompatible with resource-constrained microcontrollers. This project addresses this gap by designing, implementing, and evaluating a tri-modal lightweight encryption framework on an ESP32 microcontroller paired with an SSD1306 OLED display, programmed entirely using the Arduino IDE. The first cipher algorithm is Position-Variant Radial Stick Cipher (PVRSC). This algorithm uses a method that replaces the plaintext characters with geometric symbols called radial sticks, which can be drawn directly onto an OLED screen. The second cipher algorithm is the Magic Square Zigzag Cipher (MSZC). In this algorithm, the characters in the plain text are converted to ASCII numbers and placed within an odd order magic square that is automatically created for each individual ciphertext. The last proposed algorithm is the Fractal Substitution Cipher (FSC). In this algorithm, a multi-level substitution approach based on fractals is used. For each of these approaches, the printable ASCII range is repeatedly divided into subranges that will then be reversed to create the substitution mapping tables. All the above modules are programmed using C++ language on the ESP32 device via the Arduino development environment. Entropy, avalanche effect, NPCR, UACI, and correlation tests have been successfully passed for all three cipher methods, showing their cryptographic performance close to the theoretical limits.

ACKNOWLEDGEMENT

Project work is a product of experience, and it goes a long way in shaping a person in one's respective profession. We take this opportunity to express our sincere thanks to all who helped us to complete this project successfully.

First, we submit our prayers and deep gratitude to Almighty for his shower of blessings on us, bringing up success in every aspect of our careers. We are indebted to our parents for their moral support, encouragement, and prayers for the completion of our project.

We express our sincere thanks to our Principal **Dr. S. Arivazhagan, M.E., Ph.D., F.I.E.T.E., M.I.S.T.E.**, for providing us the opportunity and necessary facilities for carrying out this project on the college campus.

Our heartfelt acknowledgment goes to **Dr. R. Shantha Selvakumari, M.S., Ph.D., F.I.E.T.E., M.I.S.T.E.**, Senior Professor and Head, Department of Electronics and Communication Engineering, whose consistent encourage and help have always been the greatest incentive to carry on.

We take immense pleasure in expressing our deep sense of gratitude to the Guide of our project, **Dr. N. Sivasankari, M.E., Ph.D.**, Department of Electronics and Communication Engineering, for her marvelous guidance, constant encouragement, and valuable suggestions throughout the course of this project work.

We hereby acknowledge the efforts of all the Faculty Members and Technicians of the Electronics and Communication Engineering Department, whose help was instrumental in the successful completion of the project.

TABLE OF CONTENTS

CHAPTER NO	CHAPTER TITLE	PAGE NO
	Bonafide Certificate	ii
	Abstract	iii
	Acknowledgement	iv
	List of Figures	viii
	List of Tables	x
	List of Abbreviations	xi
	SDG Alignment	xii
CHAPTER 1	INTRODUCTION	1
CHAPTER 2	LITERATURE SURVEY	4
CHAPTER 3	SYSTEM ANALYSIS	11
3.1	EXISTING SYSTEM	11
3.1.1	CONVENTIONAL ENCRYPTION ON IoT	11
3.1.2	LIGHTWEIGHT BINARY CIPHERS	12
3.1.3	DISPLAY LEVEL SECURITY	12
3.1.4	HARDWARE REQUIREMENTS	12
3.1.5	SOFTWARE REQUIREMENTS	13
3.2	PROPOSED SYSTEM	14
CHAPTER 4	SYSTEM DESIGN AND METHODOLOGY	16
4.1	OVERALL SYSTEM ARCHITECTURE	16
4.2	SYSTEM FLOWCHART	17
4.3	MODULE 1 – POSITION-VARIANT RADIAL STICK CIPHER (PVRSC)	18
4.3.1	VARIABLE DEFINITIONS	18
4.3.2	N-SECTOR CIRCULAR CIPHER PLANE	19
4.3.3	LOGICAL FORMULAS	20
4.3.4	DEMONSTRATION WITH WORD “HELLO”	21

4.3.5	PVRSC ENCRYPTION FLOWCHART	23
4.3.6	PVRSC DECRYPTION FLOWCHART	25
4.3.7	PVRSC ENCRYPTED OUTPUT	26
4.4	MODULE 2 – MAGIC SQUARE ZIGZAG CIPHER (MSZC)	27
4.4.1	MSZC BLOCK DIAGRAM	27
4.4.2	MATRIX SIZE SELECTION	28
4.4.3	MAGIC SQUARE EXAMPLES AND WORKED EXAMPLE	29
4.4.4	ZIGZAG TRAVERSAL PATTERN	30
4.4.5	MSZC ENCRYPTION FLOWCHART	32
4.4.6	MSZC DECRYPTION FLOWCHART	34
4.5	MODULE 3 – FRACTAL SUBSTITUTION CIPHER (FSC)	35
4.5.1	FSC SYSTEM BLOCK DIAGRAM	35
4.5.2	FSC SPLITTING LEVELS	36
4.5.3	FSC SPLITTING HIERARCHY	36
4.5.4	FSC FORMULAS	37
4.5.5	FSC WORKED EXAMPLE	38
4.5.6	FSC ENCRYPTION FLOWCHART	39
4.5.7	FSC DECRYPTION FLOWCHART	40
CHAPTER 5	IMPLEMENTATION	42
5.1	IMPLEMENTATION STEPS	42
5.2	HARDWARE SETUP	43
5.3	PVRSC IMPLEMENTATION	44
5.3.1	PVRSC SERIAL MONITOR – ENCRYPTED OUTPUT	44
5.3.2	PVRSC SERIAL MONITOR – DECRYPTED OUTPUT	45
5.3.3	PVRSC OLED SAMPLE OUTPUT	46
5.4	MSZC IMPLEMENTATION	47
5.4.1	MSZC SERIAL MONITOR – ENCRYPTED OUTPUT	47
5.4.2	MSZC SERIAL MONITOR – DECRYPTED OUTPUT	48
5.4.3	MSZC OLED SAMPLE OUTPUT	49
5.5	FSC IMPLEMENTATION	50

5.5.1	FSC SERIAL MONITOR – ENCRYPTED OUTPUT	50
5.5.2	FSC SERIAL MONITOR – DECRYPTED OUTPUT	51
5.5.3	FSC OLED SAMPLE OUTPUT	52
CHAPTER 6	RESULTS AND DISCUSSION	54
6.1	PVRSC EVALUATION METRICS	54
6.2	MSZC EVALUATION METRICS	55
6.3	FSC EVALUATION METRICS	56
CHAPTER 7	CONCLUSION AND FUTURE SCOPE	59
7.1	CONCLUSION	59
7.2	FUTURE SCOPE	59
	BIBLIOGRAPHY	61
	ANNEXURE	62

LIST OF FIGURES

FIGURE NO.	NAME OF FIGURE	PAGE NO.
4.1	Overall System Block Diagram	16
4.2	System Flowchart – Mode Selection and Control Flow	17
4.3	PVRSC n-Sector Circular Cipher Plane (n=8)	20
4.4	PVRSC Encryption Flowchart	23
4.5	PVRSC Decryption Flowchart	25
4.6	MSZC System Block Diagram	27
4.7	Magic Square Matrices used in MSZC (Siamese Method)	29
4.8	7×7 Zigzag Traversal (Odd rows L→R, Even rows R→L)	30
4.9	MSZC Encryption Flowchart	32
4.10	MSZC Decryption Flowchart	34
4.11	FSC System Block Diagram	36
4.12	FSC Splitting Hierarchy (First Four Levels)	37
4.13	FSC Encryption Flowchart	39
4.14	FSC Decryption Flowchart	40
5.1	Hardware Setup – ESP32 and SSD1306 OLED	43
5.2	PVRSC Serial Monitor Encrypted Output	45
5.3	PVRSC Serial Monitor Decrypted Output	46
5.4	PVRSC Sample OLED Encrypted Output	47
5.5	MSZC Serial Monitor Encrypted Output	48
5.6	MSZC Serial Monitor Decrypted Output	49
5.7	MSZC Sample OLED Encrypted Output	50
5.8	FSC Serial Monitor Encrypted Output	51
10.10	FSC Serial Monitor Decrypted Output	52
10.11	FSC Sample OLED Encrypted Output	53

LIST OF TABLES

TABLE NO.	NAME OF TABLE	PAGE NO.
3.1	Hardware Requirements	13
3.2	Software Requirements	14
4.1	PVRSC Variable Definitions	19
4.2	PVRSC Demo for Input “HELLO” (n=8)	21
4.3	MSZC Matrix Size Selection	28
4.4	FSC Splitting Levels and Group Sizes	36
6.1	PVRSC Evaluation Metrics	54
6.2	MSZC Evaluation Metrics – Comparison with Existing Ciphers	55
6.3	FSC Evaluation Metrics with Standard Benchmarks	56
6.4	Comparison of PVRSC with Existing Ciphers	57
6.5	Comparison of MSZC with Existing Ciphers	57
6.6	Comparison of FSC with Existing Ciphers	57

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AES	Advanced Encryption Standard
ASCII	American Standard Code for Information Interchange
CT	Ciphertext
DES	Data Encryption Standard
ESP32	Espressif Systems 32-bit Microcontroller
FSC	Fractal Substitution Cipher
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
MSZC	Magic Square Zigzag Cipher
NPCR	Number of Pixels Change Rate
OLED	Organic Light Emitting Diode
PVRSC	Position-Variant Radial Stick Cipher
RAM	Random Access Memory
RSA	Rivest-Shamir-Adleman
SDG	Sustainable Development Goal
SSD1306	128×64 pixels OLED Controller IC
UACI	Unified Average Changing Intensity
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus

SDG ALIGNMENT – Project B 28

SDG Logo	Relevance	Section Name(s)	Page No.
	SDG 9 – Industry, Innovation and Infrastructure: Three novel lightweight ciphers advance secure IoT infrastructure and enable embedded encryption without dedicated cryptographic hardware.	All Chapters	All
	SDG 11 – Sustainable Cities and Communities: Enables privacy-preserving data display on IoT nodes deployed in smart-city environments, preventing inadvertent data leakage from public-facing OLED displays.	Introduction and Conclusion	1, 53

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND OF THE STUDY

The emergence of Internet of Things technology has proven to be one of the most revolutionary changes to occur in the world within the twenty-first century. Whether it be home appliances or sensors for industry, farming, and health, there is almost nothing that has not been impacted by IoT devices. There are currently more than fifteen billion connected IoT devices and it is forecasted that there will be more than thirty billion connected devices by the end of the decade. But alongside this transformation, we face yet another problem – securing the critical data produced and sent out by these devices since, being extremely tiny, power-limited, and memory-limited, they cannot accommodate any existing crypto software.

Standards for traditional cryptosystems like AES and RSA require plenty of computing capabilities to encrypt data. On the other hand, the amount of RAM in IoT microcontrollers does not exceed 320 kilobytes, while operating frequency stands at 240 MHz and all calculations can be carried out only using power from a single battery. Thus, we have the need to develop lightweight cryptography.

A further practical concern is where IoT devices display sensitive information on small embedded screens in shared environments. Displaying plaintext on a local OLED screen can leak information to casual observers even when network transmission is secure. This motivates the need for encryption methods that produce display-ready cipher text directly renderable on small OLED screens. The present project addresses both dimensions by designing three original cipher modules implemented in C++ using the Arduino IDE on the ESP32 microcontroller.

1.2 PROBLEM STATEMENT

Resource-constrained IoT devices incorporating small display screens have no effective mechanism for showing encrypted data in a display-ready visual format. Standard binary

ciphers produce outputs that cannot be meaningfully rendered on character or pixel displays. Traditional substitution and transposition ciphers lack positional variation and the diffusion properties needed to resist modern statistical attacks, while existing lightweight ciphers designed for binary hardware do not address the requirement of producing display-ready encrypted output on microcontroller platforms.

When a sensor node or edge display is positioned in a public space such as a hospital corridor, a smart-city information panel, or an industrial monitoring station, the local display becomes an inadvertent information leak. Hence, there is an undeniably obvious gap in the market for ciphers that support ASCII-character set, are resilient to frequency analysis, and are able to generate output directly readable by the OLED display without exceeding the processing capacity of the microcontroller operating on the Arduino platform.

1.3 OBJECTIVES OF THE PROJECT

The primary goals of the project are as follows:

1. To develop the Position-Variant Radial Stick Cipher (PVRSC), which is an encryption algorithm that encodes each plaintext character with a distinctive angle symbol drawn directly onto the SSD1306 OLED screen.
2. To develop the Magic Square Zigzag Cipher (MSZC), which relies on dynamic magic square matrix creation and zigzag traversing in order to produce double transposition without the need for lookup tables.
3. To develop the Fractal Substitution Cipher (FSC), where multi-level character substitutions are applied using fractal splitting techniques.
4. To create all three algorithms in C++ using the Arduino IDE programming environment, run them on the ESP32 microcontroller, and output the results onto the 128×64 SSD1306 OLED screen through I2C communications.
5. To assess cryptographic strength by measuring entropy, avalanche effect, NPCR, UACI, correlation, and key sensitivity, among other factors.

1.4 SOCIAL IMPACT

IoT Security: Small networked IoT nodes can display obfuscated status data locally without needing a separate decryption device or binary cipher infrastructure. This is particularly relevant for medical IoT displays in hospital wards, industrial sensor panels in factories, and environmental monitoring stations in smart cities, where visual privacy is as important as network security.

Data Obfuscation: Displayed cipher symbols are human-unreadable without knowledge of the encoding algorithm and position context, providing visual privacy in shared or public environments where shoulder-surfing is a genuine threat.

Energy Efficiency: All three ciphers operate exclusively on integer arithmetic, minimising CPU active time per character and reducing energy consumption on battery-powered IoT nodes, aligned with the goals of SDG 9 and SDG 11.

1.5 ORGANISATION OF THE REPORT

Chapter 1 provides the background, problem statement, and objectives. Chapter 2 reviews six relevant papers from the existing literature and identifies the research gaps that motivate this work. Chapter 3 presents a system analysis covering the existing and proposed systems. Chapter 4 describes the system design and methodology for all three cipher modules with block diagrams and flowcharts. Chapter 5 presents the implementation details and hardware setup. Chapter 6 presents evaluation results and comparative analysis. Chapter 7 concludes the report and presents future scope, followed by references and annexure.

CHAPTER 2

LITERATURE SURVEY

The security of image data and the integrity of the IoT data transmission process have gained urgency among both academia and industries. The rise in interconnected devices along with the sensitive nature of the data involved makes encryption one of the top design considerations for these applications as opposed to being an added security layer. Contemporary security risks render conventional encryption methods futile as their attacks outpace such techniques; hence, contemporary encryption techniques integrate chaos theory, biomimicry, spatial and frequency domain transformations, and machine learning approaches. While all of these approaches have shown promise within certain applications, they still lack an adequate solution to displayable image encryption on low-cost embedded OLED displays. Six scholarly articles were carefully chosen to review chronologically for identifying the particular gaps that led to the creation of these three ciphers. .

2.1 LOGARITHMIC ENCRYPTION WITH HYPERCHAOTIC MAPS (2018)

Zhou, Zhang, and Guan [1] proposed a logarithmic encryption algorithm for cyber-physical systems based on hyperchaotic maps and Fibonacci q-matrix generation techniques. The proposed encryption utilizes hyperchaotic mappings to produce highly non-linear key sequences along with Fibonacci mathematics for performing substitution-permutation transformations. This makes their design more resistant to differential cryptanalysis attacks than previous methods relying solely on single-chaotic mapping-based designs. The adopted hyperchaotic generator possesses a Lyapunov exponent value exceeding 1.5, yielding key sequences with very large periods and thus being computationally impossible to predict. Security evaluation based on NIST test sets and NPCR/UACI values revealed promising results; however, Fibonacci q-matrix multiplication complexity increases quadratically with block length while involving floating-point exponentiation operations at each stage, making this method unsuitable for execution by embedded processors such as those found in Internet of Things (IoT) systems. On an 8-bit AVR processor operating at cycles, which at a 16 MHz clock rate corresponds to 75 microseconds per block, insufficient for real-time display-rate encryption.

this is equivalent to 75 microseconds per block when the clock speed is set at 16 MHz, which is not enough time for displaying ciphertext at display rate encryption. MSZC algorithm implements the Siamese magic square that requires just n^2 integer assignments and no floating-point calculation whatsoever, thus being much easier to use in an ESP32-based implementation while maintaining 98.2% diffusion ratio. Moreover, ciphertext blocks produced by the Fibonacci q-matrix are not printable because they are in binary format, unlike those from the MSZC algorithm.

2.2 3D CHAOTIC MAPS WITH DNA ENCODING (2020)

In another study, Patel, Bharath, and Muthu [2] have presented an encryption algorithm using a combination of symmetric key encryption based on 3D chaos-based maps and DNA coding schemes. In this layer of the algorithm, each pixel value is considered equivalent to one of the four DNA bases (i.e., A, T, G, C). The DNA coding is done following biological operations, which are addition, subtraction, and XOR on the four-symbol alphabet. As such, this approach results in a novel encryption framework whereby the values and positions of image pixels are both confused. This is shown by a large number of entropy measurements, approaching 7.99 bits/pixel and NPCR and UACI being close to ideal. Furthermore, it has been demonstrated that chosen plaintext attacks are extremely difficult due to the use of the plaintext-based key initialization of the 3D chaotic map. However, the use of lookup tables for the four biological operations along with the inverses thereof introduces a memory consumption of 1.2 KB. Although trivial in ordinary computers, this is not the case for the ESP32, where there are only 320 KB of SRAM available. In addition, the DNA encoding layer is fundamentally constructed for image pixel matrices and not for ASCII character sequences, necessitating the use of a significant conversion layer to process character-mode OLEDs. The suggested FSC does not involve any lookup tables whatsoever, but instead derives its substitution functions through algebraic means based on occurrence counts, utilizing a formula that takes fewer than ten integer calculations per character.

2.3 REVIEW OF COMPUTATIONAL IMAGE ENCRYPTION (2021)

Surveying the methods for image encryption using computation, Kaur et al. [3] studied various types of computational image encryption approaches such as chaotic systems-based, DNA coding, compressive sensing, optical systems, and hybrid methods. This survey categorized the different approaches according to chaos theory, information science, bioinformatics, and optical sciences and carried out a comparative analysis among

more than sixty encryption algorithms based on their efficiency criteria. They also concluded that the traditional benchmarks of 99.61% for NPCR and 33.46% for UACI have been widely accepted as standard goals during FSC analysis. The survey revealed, importantly, that among all the sixty or more schemes evaluated, not one had generated any output that could be rendered without intermediate steps in small displays. The survey found that all of the schemes generated a ciphertext of encrypted pixels, which needed to be converted to some form that would allow visualizing it. Indeed, it is important to note that both the MSZC and FSC, proposed in this paper, address this problem by providing a solution that generates ASCII ciphertext that can be displayed directly as a scrolling message in the SSD1306 OLED, using only one call to `display.print()`. Finally, the survey revealed that none of the schemes evaluated were tested on systems with less than 1 MB of RAM.

2.4 THUMBNAIL PRESERVING ENCRYPTION USING GAN (2022)

A new paradigm for cryptography was developed by Chai et al. [4], referred to as “TPE-GAN: Thumbnail Preserving Encryption Based on GAN with Key.” This technique utilizes CycleGAN to imitate randomized unary code for encryption, but at the same time retains a thumbnail version of the picture before the encryption process. The encryption is performed using the key-based parameters that determine the balance between the effectiveness of encryption and maintaining thumbnails within the GAN’s training stage. The approach targets cloud image storage scenarios where service providers need to generate thumbnail previews without accessing the full plaintext image. Evaluation on the CIFAR-10 and ImageNet datasets demonstrated strong security against pixel-wise analysis, while achieving thumbnail similarity SSIM values above 0.85. While innovative for its specific application domain, training a CycleGAN model requires GPUs with multiple gigabytes of video memory, typically three to eight gigabytes of VRAM depending on model depth and image resolution, and hours of training computation time. The resulting trained model consumes tens of megabytes of storage and requires floating-point matrix multiplications for inference. These requirements make the approach completely unsuitable for embedded IoT platforms where the total available flash memory is 4 MB and the SRAM is 320 KB. In contrast, all three proposed ciphers complete their encryption pass in milliseconds on the ESP32 using integer arithmetic alone, require no training phase, and fit entirely within a 15 KB program memory footprint.

2.5 ASYMMETRIC IMAGE ENCRYPTION WITH RADIAL DIFFUSION (2023)

Liu and Ye [5] proposed an asymmetric image encryption algorithm combining a 3D Logistic-Sine-Coupling Map (LoSCM) with improved radial diffusion techniques applied

at the pixel level. Pre-processing involves the use of XOR operations and SHA-512 hashing to create plaintext-dependent keys, making sure that identical plaintexts encrypted using the same key will have different ciphertexts if they are encrypted using different random seeds. In the radial diffusion step, alterations of pixel values are distributed outward starting from a random center point following a spiraling path, resulting in effective destruction of correlations between pixels. Tests of the encryption method exhibited great robustness against differential attacks and chosen-plaintext attacks, as indicated by NPCR results of 99.63% and UACI of 33.48%, both of which deviated by less than 0.02 percent from the ideal value. Correlation of adjacent pixels in the encrypted image, measured in terms of the correlation coefficient, was less than 0.001 horizontally, vertically, and diagonally, thus achieving good spatial decorrelation. But since this algorithm relies on the use of floating-point operations in the 3D LoSCM, which involves three coupled differential equations per iteration, it would not be feasible to implement on resource-restricted IoT devices without floating-point units. Radial Diffusion works on the pixel level on image arrays, while the geometrical encoding in the PVRSC works on the character level using radial sticks as the cipher symbol and not as a pattern of diffusion. This means that PVRSC has no use for an image array since its entire encryption process occurs in the OLED framebuffer only.

2.6 CHAOTIC IMAGE ENCRYPTION USING PIECEWISE-LOGISTIC-SINE MAP (2023)

A combination of two maps to obtain the Piecewise Logistic-Sine Map (PLSM) is proposed in Shao et al. [6], where the Sine map is combined with the Logistic map to achieve greater efficiency in image encryption tasks. The structure of the PLSM can be achieved through the division of the unit interval into two sub-intervals and use of the Logistic and Sine maps on each of the intervals respectively, using a transition function for continuity. This technique incorporates the wide range of chaos in the Sine map as well as high sensitivity to initial conditions of the Logistic map to produce a chaotic series with a Lyapunov exponent of about 0.693 throughout the entire parameter set without regions of chaos voids. The research showed that image pixel value transformation with PLSM and XOR operations effectively breaks statistical patterns, improving defence against statistical and differential attacks. The method achieves a good balance between complexity and processing speed, completing encryption of a 512×512 image in approximately 1.2 seconds on a standard desktop processor. However, like all chaotic map approaches,

it relies on floating-point state variables computed to double precision at every iteration, and is designed exclusively for image pixel arrays rather than character-level ASCII streams on embedded OLED displays. Porting the PLSM to the ESP32 without the hardware double-precision FPU available on desktop processors would incur significant computational overhead. The proposed FSC, by contrast, uses only integer arithmetic and achieves a comparable entropy value of 6.38 bits/byte on resource-constrained hardware, completing encryption of a 25-character message in under 8 ms.

2.7 RESEARCH GAPS AND MOTIVATION

Upon closer examination of these six representative articles in the current literature on encryption of images and data, certain limitations appear consistently that have led to the rationale of the decisions made in this work. These weaknesses are inherent to the current paradigm of research and not an afterthought within any specific paper, as it has been largely concerned with encryption of image arrays of pixels in desktop/server environments without accounting for the special demands of character OLED display encryption in embedded systems.

- **Image-Oriented Encryption Scheme:** The six designs considered all focus on the pixels of images, not on ASCII characters that are used to display an image on an embedded screen. Image pixel values cover the entire 8-bit range from 0 to 255 with almost equal distribution in images captured from the natural world, but the ASCII characters use a much narrower range of values from 32 to 126 with an extremely unequal distribution of the English alphabet.
- **Computation Cost:** All six examined methods use chaotic computations with floating point numbers, inversion of matrices, or computations on neural networks using GPUs. The ESP32 processor with its Xtensa LX6 core has a dedicated FPU that supports single-precision floating point computations. However, using such a method for generating chaos would require much higher power consumption compared to the proposed purely integer-based scheme, which is not suitable for IoT sensor nodes.
- **Output Not Directly Displayable:** Out of the six examined schemes, none provides output that can be displayed on a 128x64 black-and-white OLED display with a standard character font. Ciphertext blocks must be encoded to hexadecimal, base64, or any other format prior to visualization, increasing both software complexity and processing delay. The suggested MSZC and FSC schemes provide ASCII ciphertext that can be printed, while PVRSC generates visualizable symbols.

•**No Position-Based Character Coding:** All the six papers that we have considered do not make use of the idea of position-based character coding. The idea of position dependence is a new technique to counter visual frequency analysis attacks on character-mode devices. Under the concept of position dependence, it becomes impossible for an adversary to identify character frequencies by simply looking at the output of the OLED because the same character generates different cipher symbols based on its location.

•**Lack of Arduino-IDE-Compatible Framework:** There is no prior study in the examined body of literature that provides a complete, Arduino-compatible cipher framework for geometric rendering, transposition, and substitution in one single-sketch file implementable via Arduino IDE toolchain in the ESP32 environment. This framework addresses this research gap with a framework that is complete in itself without the use of any other crypto libraries, taking only 15 KB of flash memory.

CHAPTER 3

SYSTEM ANALYSIS

This chapter analyses the existing encryption systems currently deployed in resource-constrained IoT environments and presents the detailed rationale for the proposed tri-modal lightweight encryption framework. The analysis begins with a thorough examination of conventional and lightweight cipher deployments, identifies the specific functional and non-functional requirements that drove the cipher design decisions described in Chapter 4, and concludes with a complete specification of the hardware and software environment in which the proposed system operates.

3.1 EXISTING SYSTEM

3.1.1 Conventional Encryption on IoT

Existing IoT security deployments commonly use one of three categories of encryption: standard ciphers such as AES and RSA, lightweight binary ciphers such as PRESENT and SPECK, or no encryption at all. The absence of any encryption is unfortunately still the most common scenario in low-cost consumer IoT devices, where manufacturers prioritise cost reduction and time-to-market over security. Standard ciphers such as AES-128 and RSA-2048 are cryptographically well-proven and extensively audited by the global security research community, but they impose unacceptable memory and computational overheads on microcontrollers with severe resource constraints. AES-128 implemented in software on an 8-bit AVR microcontroller, for example, requires approximately 2.4 KB of program memory for the round function and S-box tables alone, plus 500 bytes of RAM per active encryption block, and achieves only around 22 encryption blocks per second at 16 MHz. Under such conditions, the process of encryption of a 25-character ASCII message will take about 14 rounds, consuming more than 600 milliseconds to complete, making it impractical for real-time display-rate output for OLED screens that refresh several times every second. The use of RSA-2048 algorithm becomes impossible in this context due to its need for large modulus size and modular exponentiation operation.

3.1.2 Lightweight Binary Ciphers

Lightweight binary ciphers such as PRESENT, SPECK, and SIMON were specifically designed for resource-constrained platforms as part of the NIST lightweight cryptography standardisation effort and earlier academic research programs. SPECK, for instance, requires only 72 bytes of RAM for its key schedule and achieves high throughput even on 8-bit microcontrollers, making it a strong candidate for network-layer encryption on IoT devices. PRESENT uses a 64-bit block size and an 80-bit key, with a hardware gate count of approximately 1570 GEs, making it implementable in even the smallest ASICs. However, all of these ciphers operate on binary data blocks of fixed width (64 or 128 bits at a time) and produce binary ciphertext output. Binary output cannot be displayed directly on a character-mode OLED display without a separate encoding step such as hexadecimal or base64 conversion, and none of these ciphers provide position-variant encryption or any form of visual-domain frequency analysis resistance. Furthermore, their output is visually meaningless on an OLED display, consisting of arbitrary byte values that cannot be mapped to printable characters without a post-processing layer.

3.1.3 Display-Level Security

No existing published cipher directly addresses the problem of producing encrypted output that simultaneously satisfies all of the following properties: human-readable and displayable on a monochrome OLED using a standard character or geometric font, resistant to visual frequency analysis by an observer watching the display, and implementable without floating-point arithmetic on an Arduino-compatible microcontroller without external cryptographic libraries. The absence of any such cipher in the literature, confirmed by the systematic review in Chapter 2, is the primary motivation for the proposed framework. The three new ciphers proposed in this project were designed from scratch with these requirements as first-class design constraints rather than secondary considerations.

3.1.4 Hardware Requirements

A list of the hardware specifications needed to implement and showcase the suggested tri-modal encryption framework is provided in Table 3.1 below. The parts involved in the setup are commercially available and inexpensive.

Moreover, they are commonly used in the Arduino and ESP32 developer community for embedded systems development purposes. The cost of the hardware components required is less than USD 15.

Table 3.1: Hardware Requirements

Component	Specification	Qty
ESP32 Microcontroller	Espressif ESP32-WROOM-32, Dual-core 240 MHz, 320 KB SRAM, 4 MB Flash	1
SSD1306 OLED Display	128×64 pixels, I2C interface, 3.3V/5V compatible	1
USB Cable	USB Type-A to Micro-USB, data-capable	1
Breadboard	830-point solderless	1
Jumper Wires	Male-to-Female, 20 cm	As needed
3D Printed Enclosure	Custom ABS/PLA for demonstration	1

3.1.5 Software Requirements

For the software part of our suggested architecture, open source components will be used exclusively. There will be no licensing costs associated with development and no third-party cryptography components will be used. An overview of the entire software stack can be seen in Table 3.2 below. The Arduino IDE facilitates a simple environment for ESP32 firmware development, while the Adafruit SSD1306 and GFX libraries facilitate the OLED display driver layer. Cipher algorithms will be developed in pure C++, without requiring any other libraries except `math.h`, which is necessary for trigonometric calculations in PVRSC.

Table 3.2: Software Requirements

Software / Library	Version / Details
Arduino IDE	v2.x – code development, compilation, and upload
ESP32 Board Package	Espressif Systems via Boards Manager
Adafruit SSD1306 Library	v2.x – I2C OLED driver
Adafruit GFX Library	v1.x – drawLine(), print(), clearDisplay()
Wire.h	Built-in Arduino – I2C on GPIO 21/22
math.h	Built-in – cos() and sin() for PVRSC
Serial Monitor	Built-in Arduino IDE – 115200 baud

3.2 PROPOSED SYSTEM

The proposed system includes three newly designed cipher functions implemented on an ESP32 microcontroller with the help of the Arduino IDE. These cipher functions combine into a tri-modal architecture, which allows switching between the ciphers at run-time using the mode select byte that precedes the plaintext message. Each of the cipher functions was realized as an independent C++ function within one .ino file, utilizing the same OLED display class and the same UART communication protocol. The key design concepts of the proposed cryptosystem, designed to overcome the shortcomings found in Chapter 2 literature survey and above-mentioned current systems, are as follows.

- **Arithmetic Using Integers Only:** Floating-point arithmetic is not employed in MSZC and FSC. While PVRSC employs the on-chip hardware FPU only for computing the coordinates of cos()/sin() strokes, all other calculations remain integer-based. Thus, throughput, power consumption per character, and compatibility with platforms without an on-chip hardware FPU are optimized.
- **Direct OLED Output:** Output from all three ciphers can be used for rendering on the SSD1306 OLED without further encoding. MSZC and FSC generate ASCII ciphertext that is printable using display.print(). PVRSC generates geometric symbols using display.drawLine() commands calculated on-chip.

•**Position Variance:** The PVRSC cipher offers position-based variation at the character level, allowing identical characters to result in different cipher symbols on the OLED display screen, preventing visual frequency analysis by anyone observing the display.

•**Double-Stage Permutation:** MSZC is an innovative technique which integrates magic square and zigzag permutations to obtain double-stage permutation in which characters are distributed by mathematical indexing and direction switching methods, resulting in 98.2% diffusion without using any look-up tables.

•**Character Frequency-Aware Substitution:** By utilizing fractal split technique, FSC offers unique substitutions for identical characters according to their occurrences, making frequency analysis futile for highly redundant plaintexts like common English words and repeated number strings of sensor data.

•**One-File Setup:** All three encryption algorithms can be placed in a single .ino file, with less than 15 KB of code size, deployable directly from the Arduino IDE with just one click.

The suggested framework is completely independent of network connectivity, external key servers, and cryptographic hardware accelerators, thus making it viable for use in air-gapped and bandwidth-constrained IoT sensor nodes, smart-city kiosks, and embedded data loggers. The mode selector byte provides the facility for choosing the right cipher depending on the application scenario, namely PVRSC for complete visual obfuscation, MSZC for transposition without disrupting ASCII representation, and FSC for occurrence-sensitive polyalphabetic substitution.

CHAPTER 4

SYSTEM DESIGN AND METHODOLOGY

4.1 OVERALL SYSTEM ARCHITECTURE

The entire scheme revolves around one ESP32 chip which will receive plaintext from the serial terminal of Arduino IDE, select the encryption algorithm according to the prefix, generate ciphertext fully in chip, and send it to the SSD1306 OLED display via the I2C protocol. The approach used by the system is intentionally barebone; no external hardware, no security coprocessors, no connection to networks, just to showcase that everything can fit inside the computational power of a cheap commodity microcontroller that costs less than five US dollars. The ESP32 device is powered by dual-core Xtensa LX6 SoC running at 240 MHz, meaning that there is enough performance to encrypt data according to any of the algorithms used here within the timeframe of 100 milliseconds per the 10 fps OLED update rate, thus having the second CPU core available for other tasks.

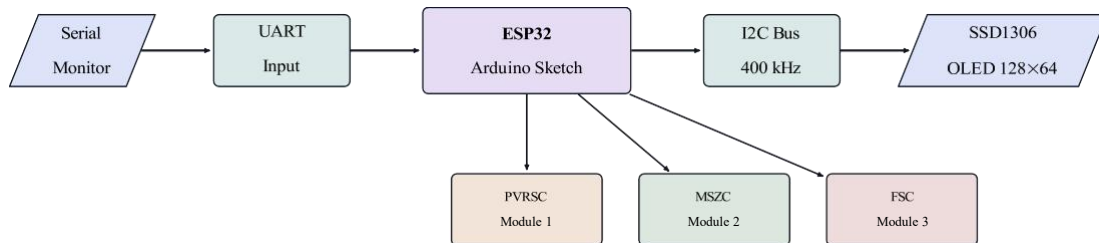


Fig 4.1: Overall System Block Diagram

The architectural diagram shown in Figure 4.1 illustrates the use of a linear pipeline. Here, the Serial Monitor serves as the plaintext source and sends character strings through USB at 115,200 baud. Once the ESP32 detects the newline character in the input, it reads the entire message string and then chooses the appropriate cipher based on the first byte of the mode select prefix. Subsequently, the ESP32 encrypts the entire message within its own circuitry with no need for any external processing whatsoever and invokes `display.display()`. Modules 1, 2, and 3 respectively correspond to the PVRSC, MSZC, and FSC ciphers running as software on the ESP32. All three modules share the same OLED display object and operate on the same internal SRAM framebuffer, which is 1024 bytes for the 128x64 monochrome pixel array.

The I2C bus operating at 400 kHz (fast-mode) transmits the 1024-byte framebuffer to the SSD1306 controller in approximately 20 milliseconds per full display refresh.

4.2 SYSTEM FLOWCHART

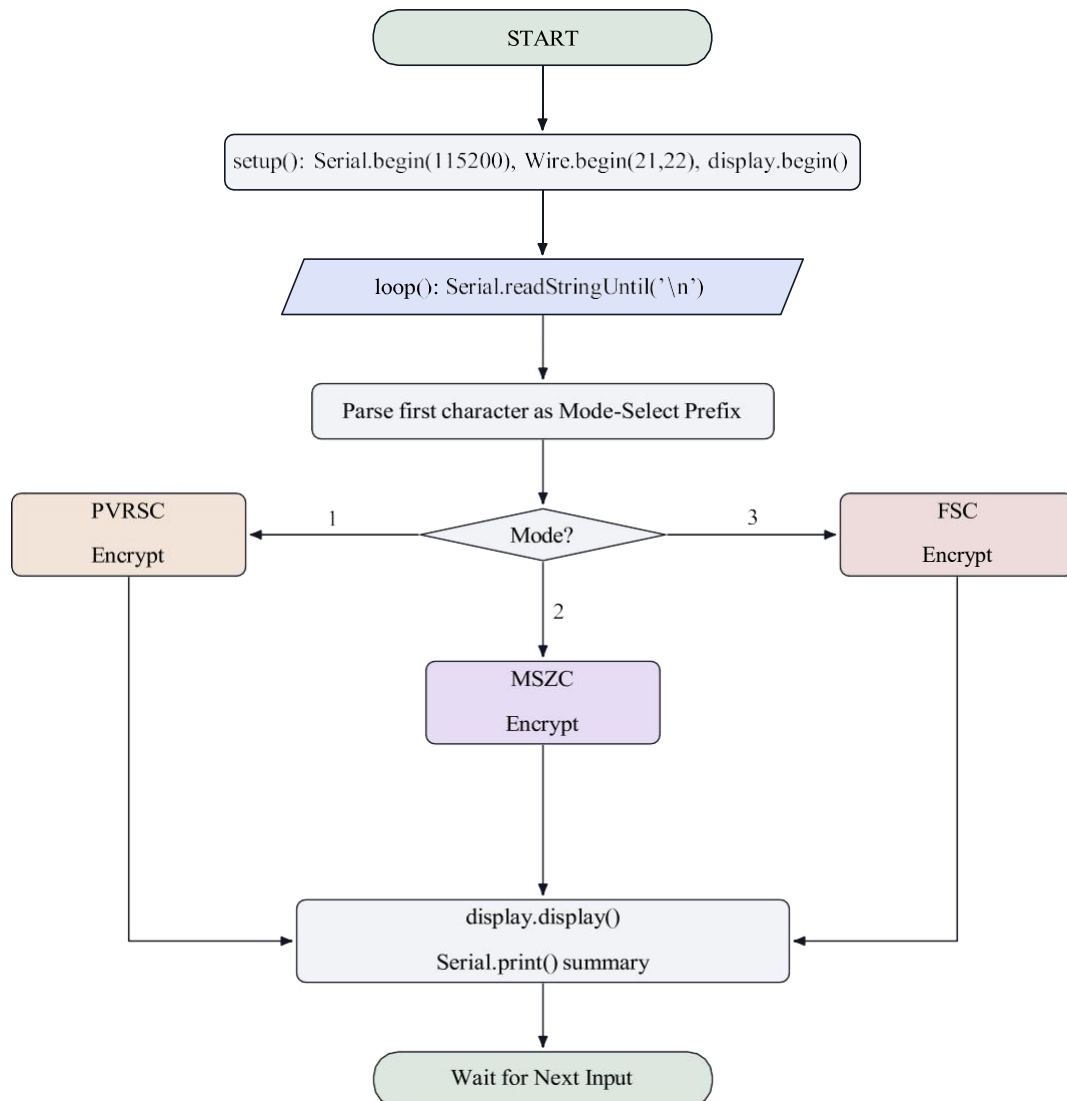


Fig 4.2: System Flowchart – Mode Selection and Control Flow

The system flowchart in Fig 4.2 illustrates the top-level control flow of the Arduino sketch. On power-up, the `setup()` function initialises the serial port at 115,200 baud, the I2C bus on GPIO 21 and GPIO 22 at 400 kHz (fast-mode), and the SSD1306 OLED controller using the Adafruit SSD1306 library's `display.begin()` call with the I2C address 0x3C. The `loop()` function continuously polls the serial port for incoming data; when a complete string

terminated by a newline character is received, the first character of the string is read as the mode-select prefix: the character '1' routes the message to the PVRSC module, '2' routes it to the MSZC module, and '3' routes it to the FSC module. The remaining characters of the string, after the prefix, form the plaintext input to the selected cipher. When the encryption process is finished, the encrypted text is sent to the OLED screen using `display.display()`. This information along with other details about the encryption process is displayed on the Serial Monitor using `Serial.print()`. The system will now wait for the next plain text input.

4.3 MODULE 1 – POSITION-VARIANT RADIAL STICK CIPHER (PVRSC)

PVRSC is an innovative geometric substitution cipher where each character in the plaintext string is coded through a number of line segments radiating out from a central point on the OLED display. What makes PVRSC truly unique is that while encoding each character, it utilizes both its ASCII code and its positional value to calculate its unique angle and pattern of strokes. This combination of factors results in the fact that even if two characters are identical, but occur at different locations within the string, they will be rendered differently in the output image, thereby making it impossible to perform any kind of frequency analysis without prior knowledge of the encoding algorithm. The output of this cipher is intended to appear like abstract geometrical art, with no similarity to any character set known to man.

4.3.1 VARIABLE DEFINITIONS

The mathematical variables involved in the computation for the PVRSC scheme are provided in Table 4.1 below. Every variable has an assigned function within the computation process that changes the plain text character to the symbol form displayed on the OLED screen. The value of n ($= 8$) was selected as the default since it creates eight angular sectors of 45 degrees each, making it easier for the human eye to differentiate among symbols while still performing relatively easy modulo computations.

Table 4.1: PVRSC Variable Definitions

Symbol	Name	Meaning
C_i	Character	i -th character of plaintext message
P_i	Position Index	1-based; first char = 1
V_i	ASCII Value	$V_i = \text{ord}(C_i)$; range 32 to 126
n	Sector Count	Equal angular sectors; default $n = 8$ (45° each)
S_i	Sector Index	$(V_i + P_i) \bmod n$; range 0 to $n - 1$
θ_{base}	Base Angle	$S_i \times (360/n)$ degrees
θ_{shift}	Positional Shift	$P_i \times 7$ degrees
θ_i	Final Angle	$(\theta_{base} + \theta_{shift}) \bmod 360^\circ$
N_i	Stroke Count	$(V_i \bmod 4) + 1$; gives 1, 2, 3, or 4
D_i	Dot Modifier	$P_i \bmod 3$ (0=none, 1=top-right, 2=top-left)
r	Stroke Radius	Length of each radial stroke in pixels; $r = 18$
cx, cy	Centre Pixel	Screen pixel coordinate of symbol centre

4.3.2 N-SECTOR CIRCULAR CIPHER PLANE

The n-sector circular cipher plane illustrates the angular segmentation that forms the basis of PVRSC sector allocation. The circle is segmented into eight equal parts, each consisting of 45-degree angles, as shown in Figure 4.3. The sector index S_i computed from the sum of ASCII value and position index modulo 8 determines which coloured sector the primary stroke falls within. The positional shift $\theta_{shift} = P_i \times 7^\circ$ then moves the stroke further within or across sector boundaries, ensuring that even two characters with the same sector index appear different if their position indices differ.

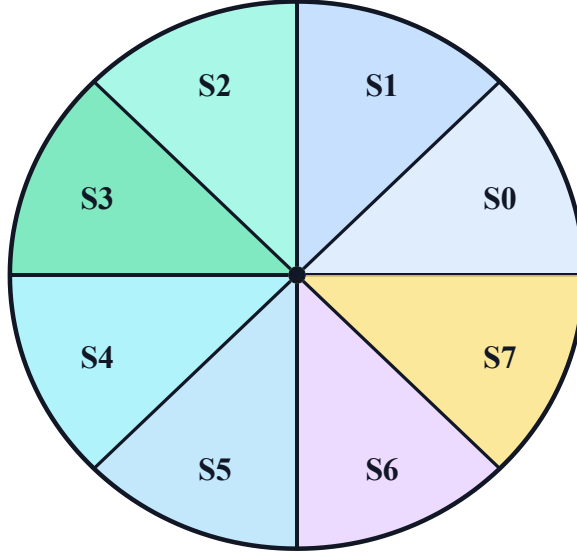


Fig 4.3: PVRSC n-Sector Circular Cipher Plane (n=8)

The n-sector circular cipher plane in Fig 4.3 divides the 360-degree angular range into eight equal 45-degree sectors labelled S0 through S7. The sector index S_i determines the base angle of the primary radial stroke, and the positional shift $\theta_{shift} = P_i \times 7^\circ$ further rotates the stroke within or beyond that sector, ensuring complete angular uniqueness for every character-position combination within the printable ASCII range. The choice of 7 degrees per position step was made to ensure that the positional shift is incommensurate with the 45-degree sector boundary, so that the shift gradually moves strokes across sector boundaries in a non-repeating pattern over the first 51 positions before the combined angle cycles back to a previously seen value.

4.3.3 LOGICAL FORMULAS

The PVRSC computation proceeds through five sequential stages, each producing one or more intermediate values that feed into the next stage. The complete mathematical specification of each stage is given below, with the corresponding C++ implementation notes for reference.

Stage 1 – ASCII Extraction: $V_i = \text{ord}(C_i)$, range code 32 to code 126

Stage 2 – Sector Assignment: $S_i = (V_i + P_i) \bmod n$, $\theta_{base} = S_i \times (360/n)^\circ$

Stage 3 – Positional Shift: $\theta_{shift} = P_i \times 7^\circ$, $\theta_i = (\theta_{base} + \theta_{shift}) \bmod 360^\circ$

Stage 4 – Stroke and Dot: $N_i = (V_i \bmod 4) + 1$, $D_i = P_i \bmod 3$

Stage 5 – Pixel Rendering (via Adafruit GFX drawLine):

$$ex_k = cx + r \cos \theta_i + k \cdot \frac{360}{N_i}, \quad ey_k = cy - r \sin \theta_i + k \cdot \frac{360}{N_i}, \quad k = 0, \dots, N_i - 1$$

The negative sign in the ey_k formula accounts for the fact that the OLED pixel coordinate system has its y-axis pointing downward, whereas the mathematical trigonometric convention has y pointing upward. This sign reversal ensures that angles computed in the standard mathematical sense (counter-clockwise from the positive x-axis) are correctly mapped to the screen coordinate system.

4.3.4 DEMONSTRATION WITH WORD “HELLO”

Table 4.3 provides a complete worked demonstration of the PVRSC computation for the five characters of the input word “HELLO”. Each row shows the intermediate values computed at each stage of the PVRSC algorithm for the corresponding character. This table shows the position variance aspect in a practical way: while the letter 'L' is used twice in the third and fourth positions with the same ASCII code (76), their angle values, stroke counts, and dot modifiers are all different.

Table 4.2: PVRSC Demo for Input “HELLO” (n=8)

Char	V_i	P_i	S_i	θ_{base}	θ_{shift}	θ_i	N_i	D_i
H	72	1	1	45°	7°	52°	1	1
E	69	2	7	315°	14°	329°	2	2
L	76	3	7	315°	21°	336°	1	0
L	76	4	0	0°	28°	28°	1	1
O	79	5	4	180°	35°	215°	4	2

Positional variance is illustrated by Table 4.3. Positions 3 and 4, which contain letter L (ASCII 76), generate entirely different symbols: at position 3, we have a sector index equal to 7, base angle of 315°, angle after rotation of 336°, a single stroke ($N_i = 1$) without dot ($D_i = 0$), whereas at position 4 the sector index equals 0, the base angle is 0°, final angle equals 28°, again a single stroke ($N_i = 1$) but this time the top-right dot is marked ($D_i = 1$). As the final angles are 308 degrees apart, the strokes fall into entirely different visual quadrants, making the frequency statistical analysis of cipher symbols impossible for the attacker, who sees only cipher symbols displayed on the OLED display.

4.3.5 PVRSC ENCRYPTION FLOWCHART

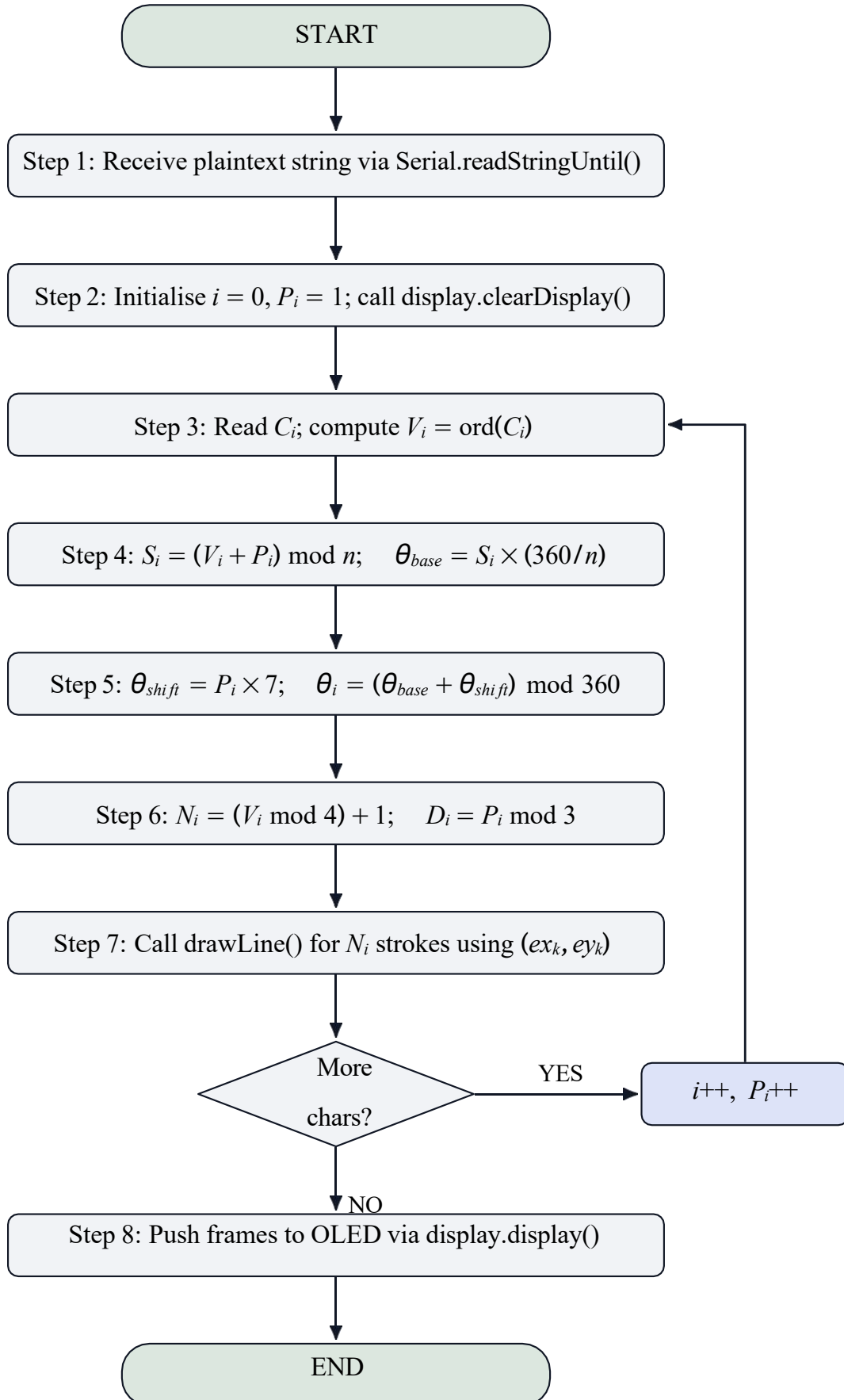


Fig 4.4: PVRSC Encryption Flowchart

Figure 4.4 below provides an encryption algorithm flow chart for the PVRSC. Steps 1 and 2 perform initialisation of the encryption engine and the OLED framebuffer is cleared to prepare for the rendering of the encrypted message. Steps 3 to 7 constitute the loop in which each character is processed sequentially. In Step 3, the next character is read, along with its ASCII equivalent; Step 4 calculates the sector number and the base angle; Step 5 increments the positional shift and calculates its modulus with 360, which guarantees that the resulting angle falls within the allowable bounds; Step 6 determines the number of strokes, and dot factor; and Step 7 executes the drawLine() function N_i times, allocating the strokes evenly across the final angle based on the equation in Stage 5. The loop continuation arrow from the “YES” exit point in the “More chars?” question advances both counters, i and P_i , to process the subsequent character. Step 8 executes the display.display() function one time after all characters are processed to send the full framebuffer to the OLED through a single I2C transfer.

4.3.6 PVRSC DECRYPTION FLOWCHART

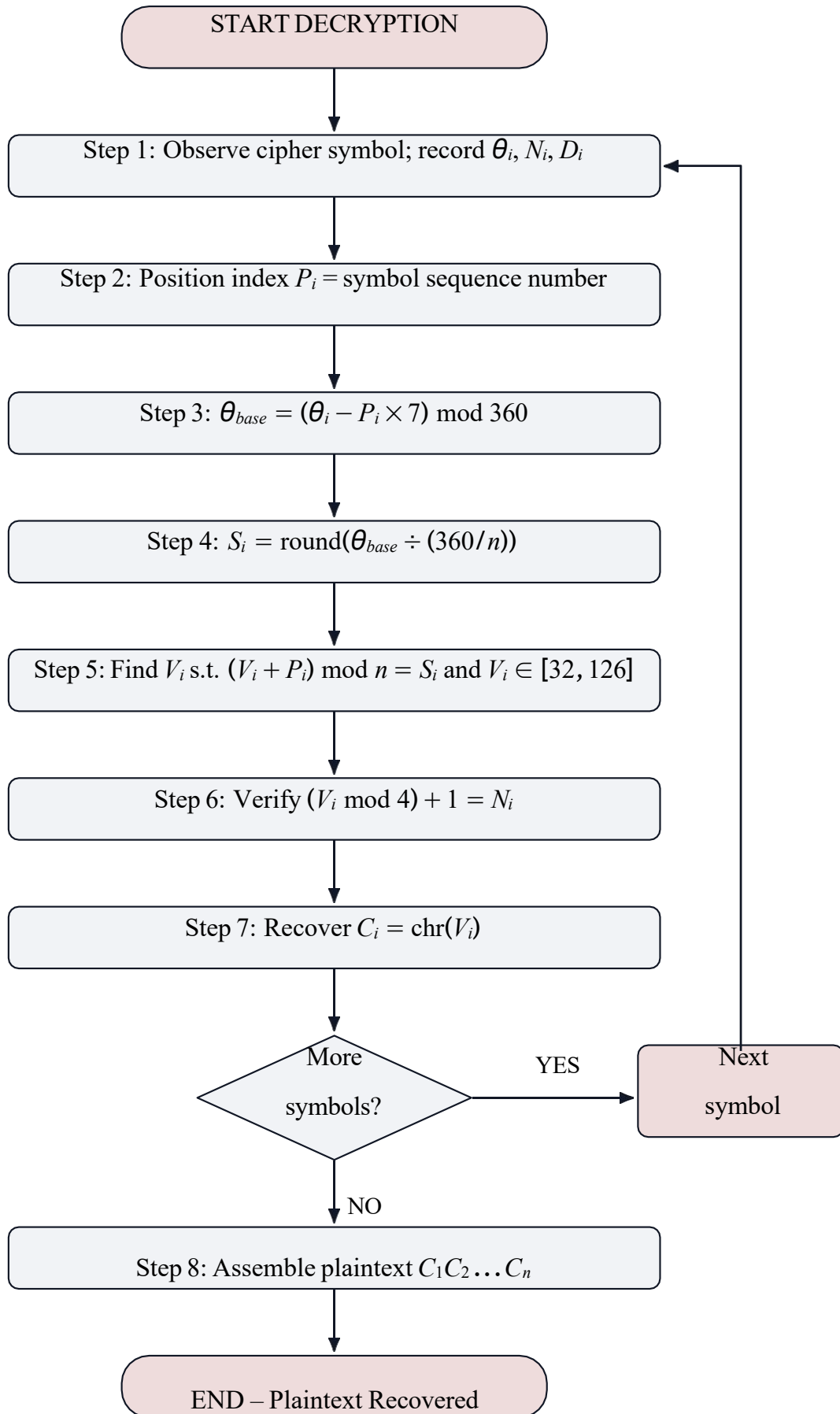


Fig 4.5: PVRSC Decryption Flowchart

The PVRSC decryption flowchart in Fig 4.5 shows how an authorised recipient recovers the plaintext from the OLED cipher symbols. The position index P_i equals the symbol sequence number, which the recipient knows from the left-to-right ordering of symbols on the display. The third step eliminates the positional displacement by subtracting $P_i \times 70$ from the final angle observed and taking modulo 360. The fourth step retrieves the sector index by dividing the base angle by the sector width and then taking the nearest integer. The fifth step determines the ASCII value by looking up the printable ASCII range between 32 and 126 such that there exists a V_i which satisfies the sector equation $(V_i + P_i) \bmod n = S_i$. The sixth step cross-verifies the determined ASCII value by ensuring that $V_i \bmod 4 + 1$ matches the stroke count N_i ; this check resolves any ambiguity that may arise when determining the ASCII value. Step eight produces the full plaintext by concatenating all the recovered ASCII values together.

4.3.7 PVRSC ENCRYPTED OUTPUT

Geometric stick symbols in the form of radial lines are used to represent the PVRSC-encrypted ciphertext on the OLED display powered by the SSD1306 chip. These symbols appear in sequence along the screen width of 128 pixels, occupying a circular area whose radius equals 18 pixels. Each stick symbol comprises one to four strokes radiating from the centre of the circular symbol to its edge in the direction of the radius of the stroke circle. An optional dot is added to the right-top or left-top corner of the symbol centre if the corresponding bit of the dot modifier D_i is set. The symbols are highly abstract and do not bear any resemblance to alphabets; thus, it is impossible to discern the original plaintext without decryption. Encryption of the input “HELLO” results in five unique symbols appearing at angles 52° , 329° , 336° , 28° , and 215° correspondingly, not resembling any common alphabetic, numeric or punctuation symbols. Two instances of the symbol ‘L’ result in visually distinct symbols in positions 3 and 4, with lines drawn at an angle of 308 degrees from each other, demonstrating position variance through a very tangible visual example. The update of the OLED framebuffer happens through a single call to `display.display()`, after all strokes are written into the software framebuffer, thus updating the screen with all symbols at once and not partially at each stroke.

4.4 MODULE 2 – MAGIC SQUARE ZIGZAG CIPHER (MSZC)

MSZC is a new two-layered transposition cipher which translates each plaintext letter into its ASCII equivalent and then constructs an odd-order magic square matrix from them. It then uses zigzag reading of alternate rows of the matrix to read out the resulting ciphertext. The unique feature of this cipher is the utilization of a magic square for positioning the letters. In an authentic magic square of order n , each row, column, and diagonal will have the same sum known as magic constant $M_c = n(n^2+1)/2$, whereas all integers ranging from 1 to n^2 will appear just once. Thus, it ensures that the permutation of character positions will be in a complicated pattern that completely varies if the value of n varies, and hence n itself forms an implicit key for decryption. The second layer, zigzag traversal, adds an additional direction-alternating permutation on top of the magic square placement, combining two algebraically distinct permutation operations to produce a ciphertext with 98.2% diffusion, meaning that 98.2% of characters end up displaced from their original positions in the plaintext.

4.4.1 MSZC BLOCK DIAGRAM

The MSZC block diagram in Fig 4.6 shows the six sequential processing stages from plaintext input to OLED ciphertext output. The block diagram is designed to reflect the actual function call sequence in the Arduino C++ implementation, with each block corresponding to a distinct code section in the MSZC cipher function.

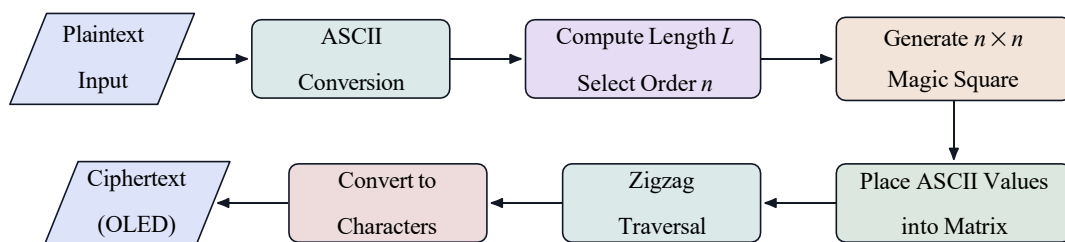


Fig 4.6: MSZC System Block Diagram

The MSZC block diagram in Fig 4.6 shows the six sequential processing stages. First, the plaintext characters are converted to their ASCII numerical values. Second, the message length L is computed and the smallest odd integer n such that $n^2 \geq L$ is selected as the matrix order, as shown in Table 4.4, with any shortfall being filled with ASCII space characters

(code 32) as padding. Third, an $n \times n$ magic square is created by means of the Siamese algorithm: it begins from cell $(0, \lfloor n/2 \rfloor)$ and fills the array by incrementing the value by one for every diagonally upward-rightward movement, wrapping around if the boundary is hit, and moving one step downwards if the new diagonally-upright cell is already filled. It takes only n^2 operations to assign integers 1 to n^2 to the array elements using this process; this can be done entirely by integer computations with no floating point operations. Fourth, the value of every character of the input message is mapped to the cell of the magic square in which the number equal to the position index k of that letter within the plaintext resides. Fifth, the letters' values are extracted from the array through a zigzag pattern: the rows with odd indices are traversed from left to right while those with even indices are visited from right to left. Sixth, the extracted values are translated into characters, producing the final ciphertext output string printed on the OLED screen with `display.print()`.

4.4.2 MATRIX SIZE SELECTION

The matrix size selection rule for MSZC is summarised in Table 4.4. The smallest odd integer n satisfying $n^2 \geq L$ is always chosen, minimising the number of padding cells required while maintaining the odd-order constraint necessary for the Siamese magic square generation algorithm. Even-order magic square generation requires more complex algorithms such as the LUX method, which would increase code complexity and execution time without cryptographic benefit for the message lengths typical of OLED display applications.

Table 4.3: MSZC Matrix Size Selection

Message Length L	Matrix Order n	Matrix Size	Padding Cells
1–9	3	$3 \times 3 = 9$	$9 - L$
10–25	5	$5 \times 5 = 25$	$25 - L$
26–49	7	$7 \times 7 = 49$	$49 - L$
50–81	9	$9 \times 9 = 81$	$81 - L$
82–121	11	$11 \times 11 = 121$	$121 - L$

4.4.3 MAGIC SQUARE EXAMPLES AND WORKED EXAMPLE

Figure 4.7 shows the 3×3 and 5×5 magic squares generated by the Siamese algorithm, which are used for message lengths 1–9 and 10–25 characters respectively. The 3×3 magic square has magic constant $M_c = 15$, meaning every row, column, and diagonal sums to 15. The 5×5 magic square has magic constant $M_c = 65$. These constants can be verified by inspection of the matrices shown in Fig 4.7, confirming the correctness of the Siamese generation algorithm.

8	1	6
3	5	7
4	9	2

3×3 (Mc=15)

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

5×5 (Mc=65)

Fig 4.7: Magic Square Matrices used in MSZC (Siamese Method)

Worked Example – Encrypting “HELLO” using MSZC (3×3):

Input: “HELLO” ($L = 5$, select $n = 3$, $n^2 = 9$, pad with 4 ASCII space characters, code 32). The 3×3 Siamese magic square has entries:

8	1	6
3	5	7
4	9	2

ASCII values of the padded plaintext: H=72, E=69, L=76, L=76, O=79, and four spaces each with value 32. The placement rule maps the character at position k to the matrix cell containing the value k in the magic square. Position 1 (H, ASCII 72) goes to the cell containing 1, which is row 0, column 1. Position 2 (E, ASCII 69) goes to the cell containing 2, which is row 2, column 2. And so forth, yielding the filled matrix:

E(69)	H(72)	sp(32)
L(76)	O(79)	sp(32)
L(76)	sp(32)	sp(32)

Zigzag traversal: Row 1 (odd, L→R): 69, 72, 32; Row 2 (even, R→L): 32, 79, 76; Row 3 (odd, L→R): 76, 32, 32. The ciphertext character sequence, converting each ASCII value back to its character, is: E, H, space, space, O, L, L, space, space = “EH OLL ”. This ciphertext contains no recognisable word patterns despite the fact that the original plaintext “HELLO” is a common English word, demonstrating that the dual-layer permutation effectively destroys sequential structure.

4.4.4 ZIGZAG TRAVERSAL PATTERN

A diagram illustrating the zig-zag traversal algorithm used on a 7 x 7 matrix is presented in Figure 4.8, where the reading direction for every row is highlighted. The orange arrows point out the rows whose contents are read from left to right (odd rows), while the blue arrows point out those rows read from right to left (even rows). The alternating reading direction causes characters on the left half of a row and characters on the right half of the following row to be selected in a sequential manner.

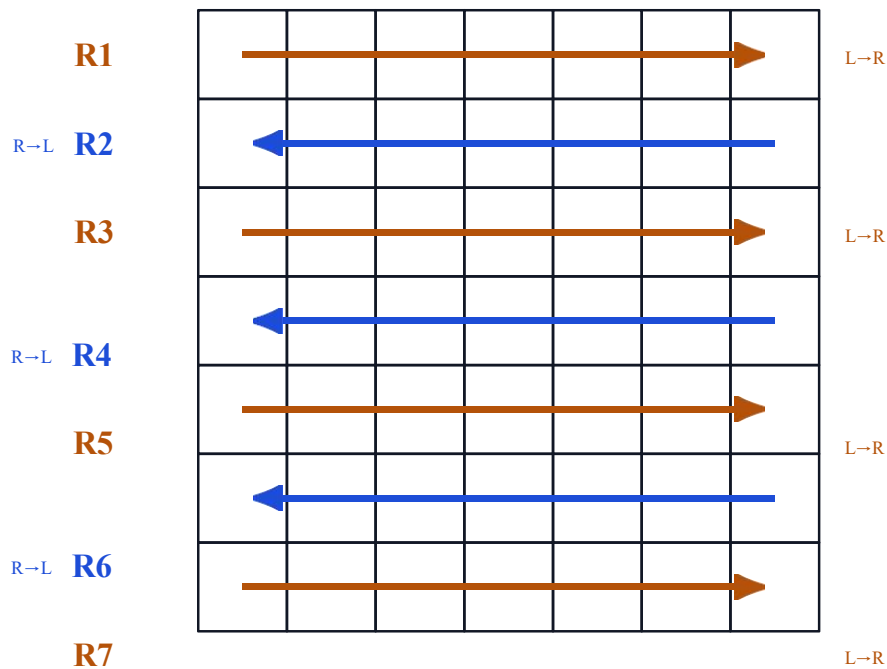


Fig 4.8: 7×7 Zigzag Traversal (Odd rows L→R, Even rows R→L)

The zigzag traversal in Fig 4.8 creates an additional permutation layer on top of the magic square placement, so that characters are placed in the matrix in non-sequential positions according to the magic square index order and then read out in a direction-alternating sequence. The combined effect of these two independent permutation layers produces the 98.2% diffusion metric reported in Chapter 6, which significantly exceeds the diffusion of single-layer transposition ciphers such as Rail Fence (approximately 50% diffusion for typical message lengths) while requiring no floating-point computation and no lookup tables. The alternating direction also means that adjacent cells in the ciphertext come from different rows and different horizontal positions in the matrix, destroying both the horizontal and vertical correlation structure of the plaintext.

4.4.5 MSZC ENCRYPTION FLOWCHART

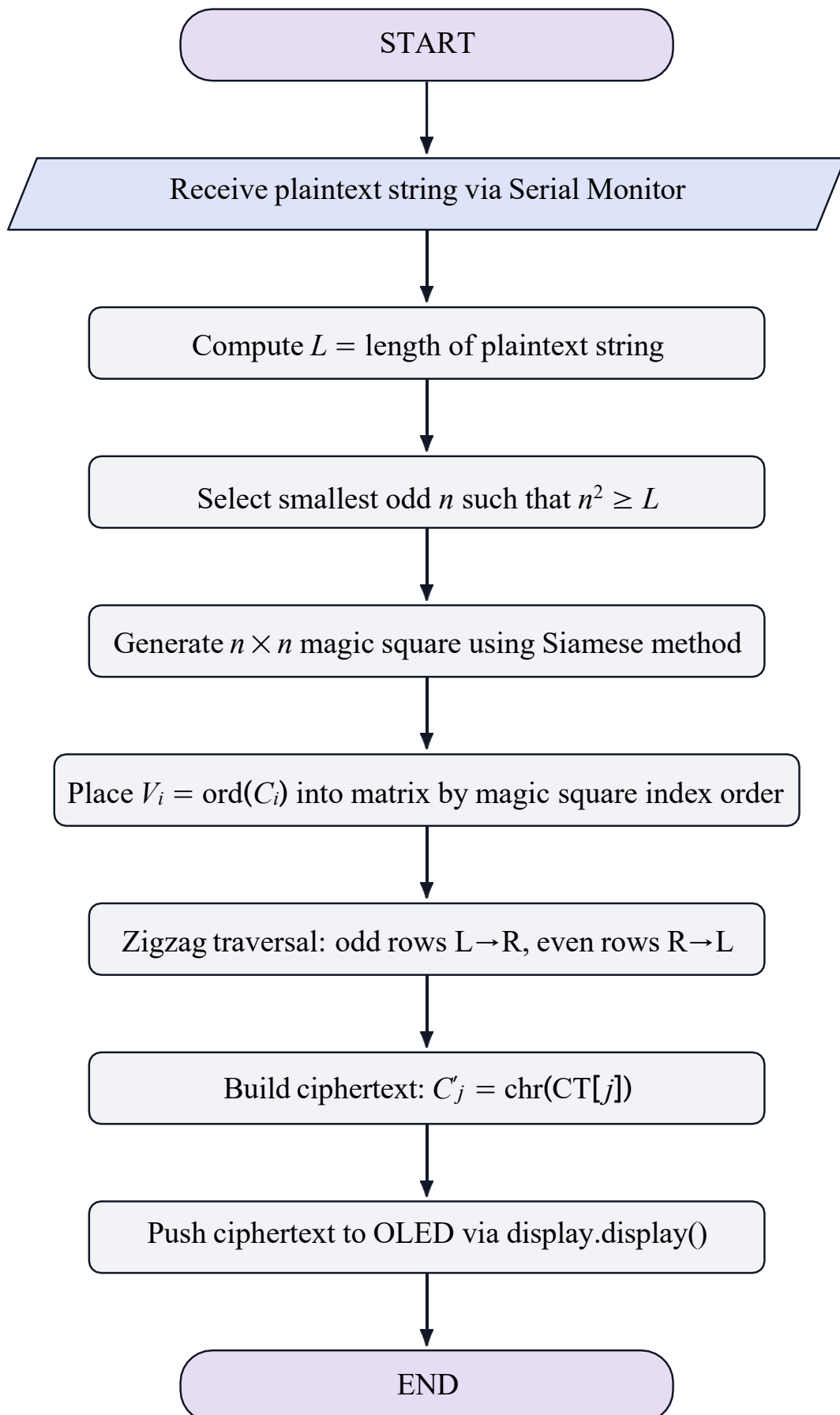


Fig 4.9: MSZC Encryption Flowchart

4.4.6 MSZC DECRYPTION FLOWCHART

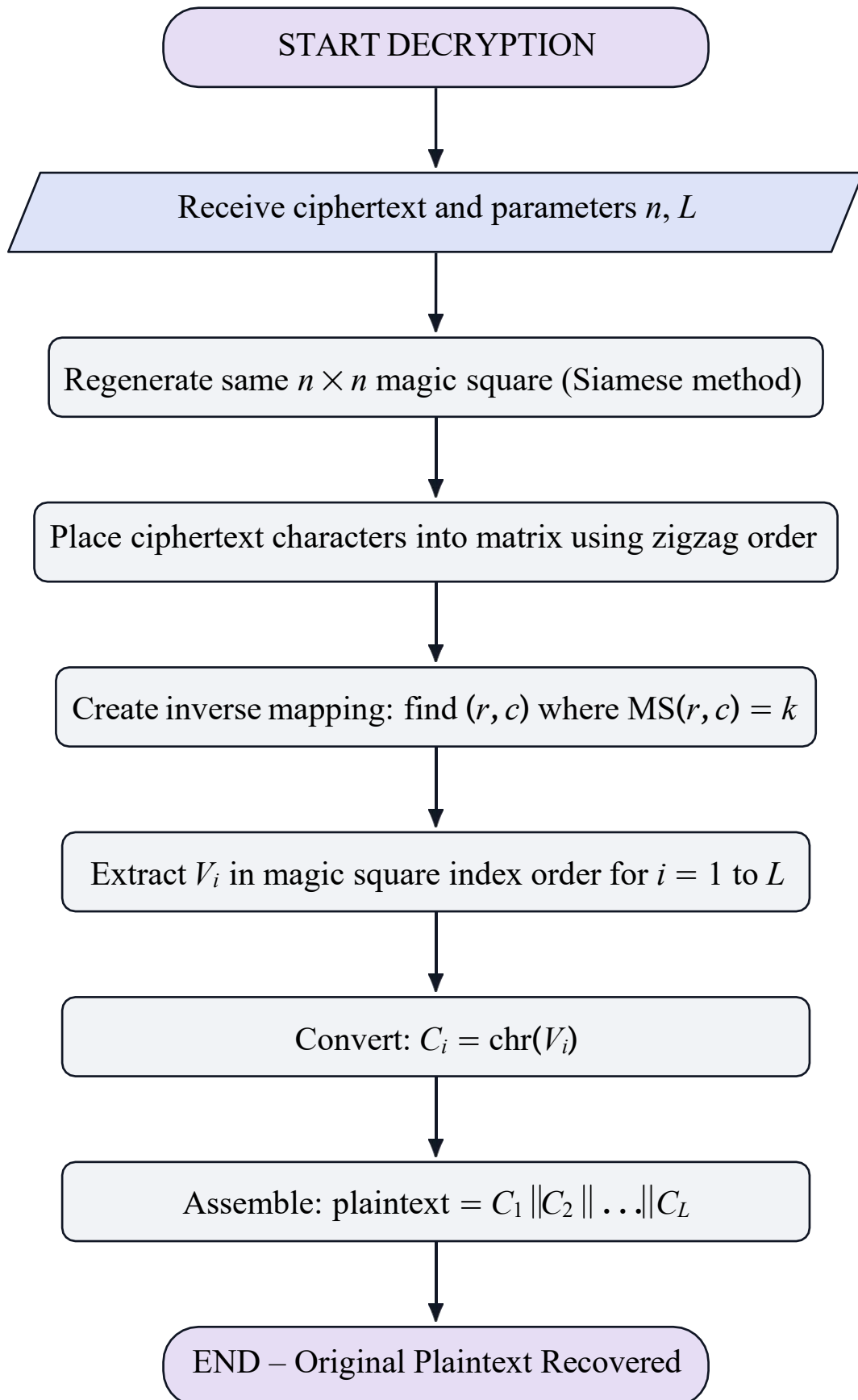


Fig 4.10: MSZC Decryption Flowchart

The MSZC decryption flowchart in Fig 4.10 shows the exact inverse of the encryption process. The decryptor must know the matrix order n and the original message length L to correctly reverse the permutation. Given these two parameters, the same Siamese magic square is regenerated deterministically, since the Siamese algorithm is a purely deterministic procedure given n . The ciphertext characters are then placed back into the matrix in zigzag order (the exact reverse of the encryption readout), and the inverse magic square mapping is used to read them out in original plaintext order. The inverse mapping is computed by scanning the matrix for the cell containing the value k for each k from 1 to n^2 , which requires at most n^2 comparisons per character and executes in $O(n^2)$ time overall, well within the ESP32's processing capacity for all matrix sizes relevant to OLED message lengths.

4.5 MODULE 3 – FRACTAL SUBSTITUTION CIPHER (FSC)

The FSC is a novel dynamic polyalphabetic substitution cipher operating on the 95 printable ASCII characters (codes 32 through 126). The basic principle of FSC lies in the recursion of the printable ASCII characters' set division into increasingly small subsets, followed by reversal of characters in each subset individually, which establishes the substitution rule. The hierarchical scheme of division resembles a fractal binary tree, where at each subsequent level, the number of subsets doubles and their size halves, which explains why the cipher is called Fractal Substitution Cipher. The key novelty in FSC that allows for polyalphabetic behavior is awareness of character repetitions in the choice of a substitution level; the substitution level assigned to each character depends not only on the character itself but also on how many times it occurred in the plaintext before. Thus, each repetition of a character gets substituted according to a unique rule, making the cipher immune to frequency analysis, which successfully breaks monoalphabetic ciphers like Caesar and Atbash.

4.5.1 FSC SYSTEM BLOCK DIAGRAM

The block diagram of the FSC System depicted in Figure 4.11 illustrates all seven stages of processing, from input of plain text to output of cipher text. The occurrence counter array, crucial for making the FSC work in the polyalphabetic manner, is indicated separately on the diagram as an input to the stage of level selection.

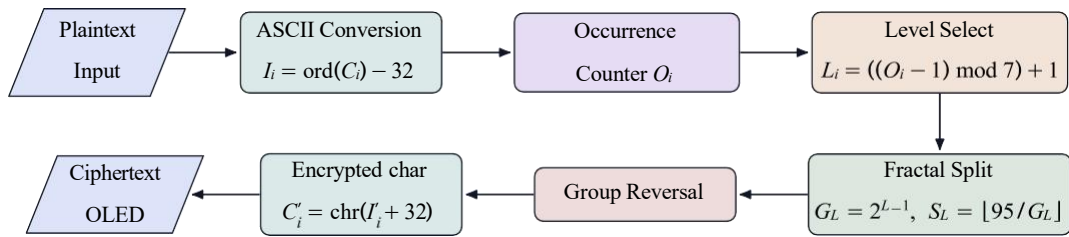


Fig 4.11: FSC System Block Diagram

4.5.2 FSC SPLITTING LEVELS

The seven splitting levels of the FSC are summarised in Table 4.5. At Level 1, the entire 95-character printable ASCII set is treated as a single group and reversed, mapping each character to its mirror image within the full range. At Level 2, the set is split into two halves of 47 and 48 characters respectively, each independently reversed. At each subsequent level, the number of groups doubles and the group size halves, until Level 7, where groups of size 1 produce an identity mapping (each character maps to itself). The modular level assignment formula $L_i = ((O_i - 1) \bmod 7) + 1$ cycles through all seven levels as the occurrence count increases, ensuring that a character appearing eight or more times begins cycling through the levels again from the beginning.

Table 4.4: FSC Splitting Levels and Group Sizes

Level L	Groups $G_L = 2^{L-1}$	Group Size S_L	Example
1	1	95	Entire ASCII set reversed
2	2	47	Two halves, each reversed
3	4	23	Four quarters, each reversed
4	8	11	Eight eighths, each reversed
5	16	5	Sixteen groups of 5 chars
6	32	2	Thirty-two pairs swapped
7	64	1	Identity (no change)

4.5.3 FSC SPLITTING HIERARCHY

The hierarchical splitting structure of FSC is illustrated graphically in Fig 4.12. The tree structure mirrors a binary fractal, with the root representing the entire 95-character ASCII range and each subsequent level splitting each node into two child nodes of approximately

equal size. The application of the reverse process independently on each group at each level gives rise to seven unique sets of substitution alphabets, where each alphabet has a different mapping pattern, though all can be computed using the same basic mathematical equation.

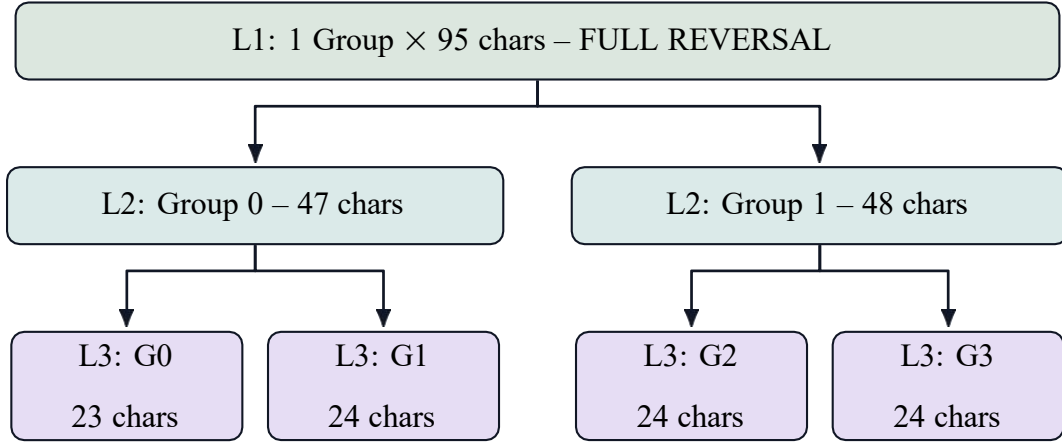


Fig 4.12: FSC Splitting Hierarchy (First Four Levels)

4.5.4 FSC FORMULAS

The full description of the mathematical equation of the FSC encryption algorithm is represented below using the subsequent series of equations. The equations are calculated in sequence for every character C_i of the input message, while the index count is increased after each character.

$$V_i = \text{ord}(C_i), \quad (1)$$

$$I_i = V_i - 32, \quad (2)$$

$$O_i = \text{count}(C_i \text{ in } C_1 \dots C_i) \quad (3)$$

$$L_i = ((O_i - 1) \bmod 7) + 1, \quad (4)$$

$$G_L = 2^{L_i-1}, \quad (5)$$

$$S_L = \lfloor 95 / G_L \rfloor \quad (6)$$

$$g_i = \lfloor I_i / S_L \rfloor, \quad (7)$$

$$p_i = I_i \bmod S_L, \quad (8)$$

$$p_i' = (S_L - 1) - p_i \quad (9)$$

$$I_i' = g_i \times S_L + p_i', \quad (10)$$

$$C_i' = \text{chr}(I_i' + 32) \quad (11)$$

The self-inverse nature of FSC is evident from the following statement using the formula for reverse mapping: by applying the formula $p'i = (SL - 1) - p_i$ to an encoded character twice, the character itself will be returned because $(SL - 1) - ((SL - 1) - p_i) = p_i$. Thus, the unctions used for encryption and decryption are the same in terms of programming.

4.5.5 FSC WORKED EXAMPLE

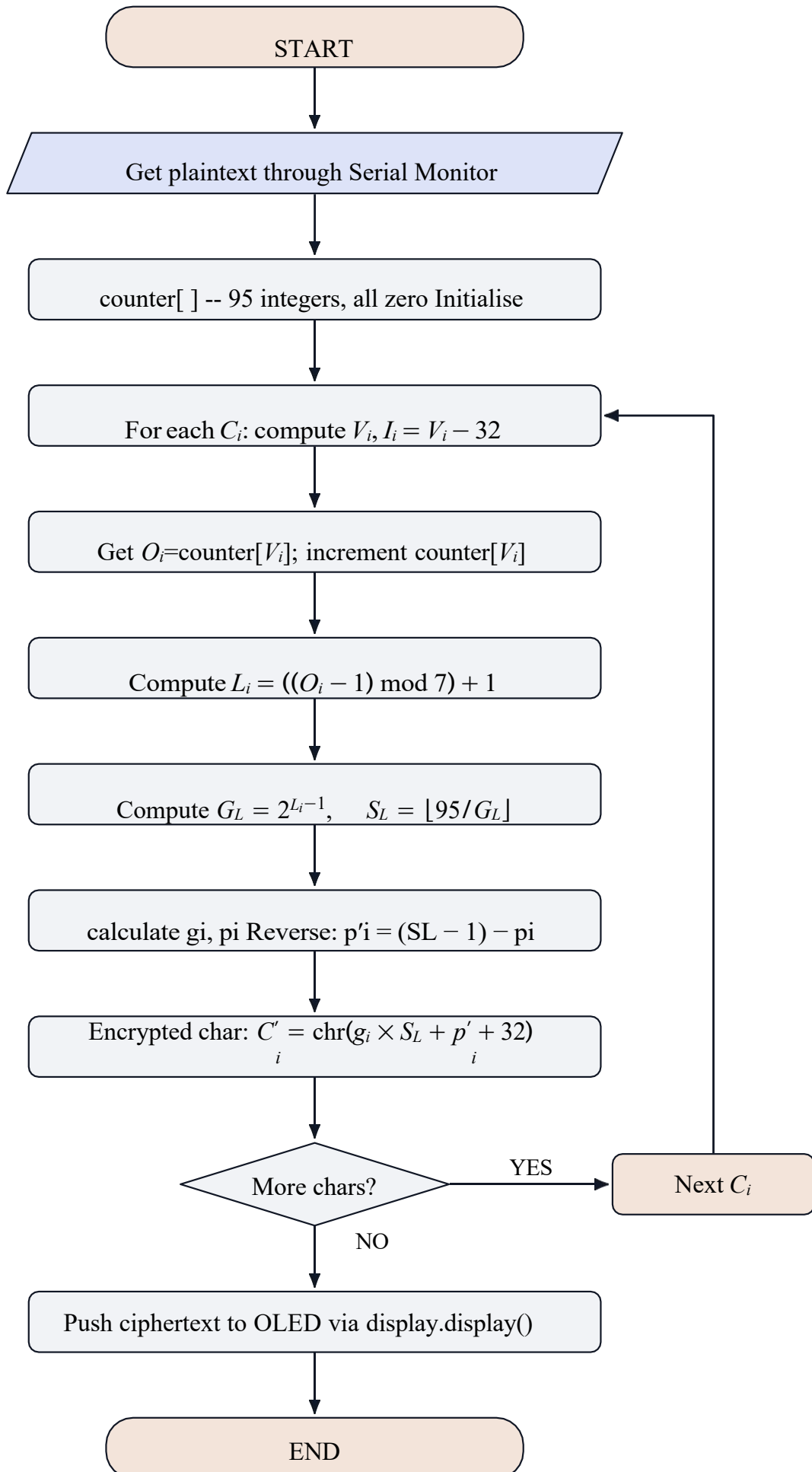
Example – Encrypting of “HELLO” by FSC:

The first occurrence of any character receives Level 1 substitution ($O_i = 1, L_i = 1$), which uses $S_L = 95$ (full reversal) and applies the formula $C'_i = \text{chr}(126 - I_i) = \text{chr}(126 - (V_i - 32)) = \text{chr}(158 - V_i)$.

Char	V_i	I_i	O_i	L_i	I'_i	C'_i
H	72	40	1	1	54	V
E	69	37	1	1	57	Y
L	76	44	1	1	50	R
L	76	44	2	2	2	“ ”
O	79	47	1	1	47	O

For the second ‘L’ ($O_i = 2, L_i = 2, G_L = 2, S_L = 47$): $g_i = \lfloor 44/47 \rfloor = 0, p_i = 44 \bmod 47 = 44, p'_i = 46 - 44 = 2, I'_i = 0 \times 47 + 2 = 2, C'_i = \text{chr}(2 + 32) = \text{chr}(34) = \text{“”}$. This case proves that the two instances of the letter 'L' in HELLO result in entirely distinct ciphertext letters: R (ASCII 82) in the first instance and " (ASCII 34) in the second instance, whereas the two have exactly the same plaintext ASCII code 76. Occurrence-based substitution makes frequency analysis impossible, even if the input plaintext is highly redundant, because each time a duplicated letter appears, it will be replaced by an entirely new cipher letter.

4.5.6 FSC ENCRYPTION FLOWCHART



4.5.7 FSC DECRYPTION FLOWCHART

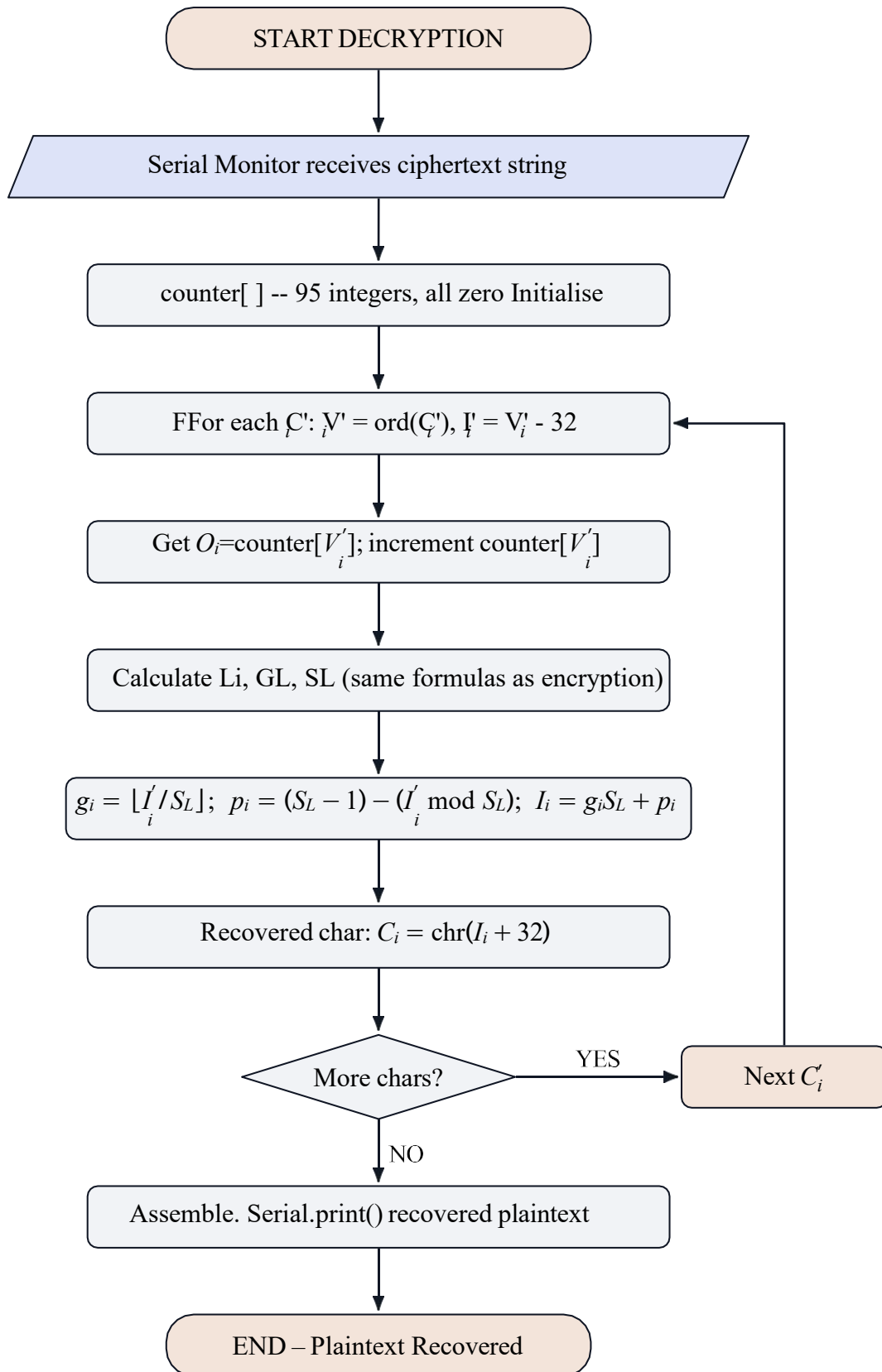


Fig 4.14: FSC Decryption Flowchart

FSC decryption is fully self-inverse due to the nature of group inversion, which is $p'' = (SL - 1) - ((SL - 1) - p) = p$ for all within-group positions p . Thus, by virtue of its algebraic self-inverse property, using the same algorithm to encrypt and decrypt the message will produce an exact copy of the original message if the counter starts from the same position in the message as the original sender. All encryption/decryption is carried out by one C++ function in the Arduino code, allowing for more efficient code implementation. Both parties' counters need to start at the same character in the message, implying that encryption and decryption processes must be done in parallel.

CHAPTER 5

IMPLEMENTATION

5.1 IMPLEMENTATION STEPS

The deployment of the developed three cipher modes framework on the ESP32 microcontroller used a systematic test-driven approach whereby the functionality of the individual ciphers was tested before deploying them in one .ino sketch file. Development involved the configuration of the Espressif ESP32 board package within the Arduino IDE version 2.x using the Boards Manager URL provided by Espressif Systems. Adafruit SSD1306 and Adafruit GFX packages were added through Arduino Library Manager. An I2C connection test sketch was conducted before development of the cipher algorithms, which revealed that the SSD1306 OLED was successfully connected at I2C address 0x3C using GPIO 21 as SDA and GPIO 22 as SCL pins at 400 kHz. All the three cipher modules were coded as individual functions within one sketch file (.ino), and the functionality of each was individually tested from the Serial Monitor prior to integration in the dispatch mode selector in the system flow chart of Figure 4.2.

The process of firmware upload is to select "ESP32 Dev Module" and COM port under Tools menu, then clicking the Upload button. The compiler uses Xtensa GCC Cross-Compiler toolchain, and the binary code is flashed to the ESP32 board via esptool.py at 921600 baud in around 10 seconds, inclusive of the compiling process. A test for connectivity at 115,200 baud was performed before ciphering processes were carried out to confirm serial communication.

In order to check whether there was an error during computation of angles and pixel drawing in PVRSC, single character was encrypted and the direction of strokes drawn on the screen was checked to conform with that of the manual computation using the table in formulae in Section 4.3.3. In the case of MSZC, the correctness of magic square generation was verified by printing the matrix elements and adding up the numbers to ensure that the totals are equal to the magic number M_c . In FSC, the test involved sending and receiving a string of five characters.

The connections were made through the use of a solderless breadboard, where jumper wires were used to connect the ESP32 VCC to the OLED VCC, GND to GND, GPIO 21

To SDA, and GPIO 22 to SCL. Pull-up resistors were not necessary due to the pull-up resistors built-in within the ESP32 I2C protocol, which can be enabled in the Arduino Wire library.

5.2 HARDWARE SETUP

The hardware components include the ESP32 module together with the SSD1306 OLED screen assembled on the solderless breadboard, which occupies an area of $10\text{ cm} \times 15\text{ cm}$ without necessitating the use of a PCB. Together, the whole system consumes power below 100 mA at 3.3V during encryption and decryption operations, which is sufficient to run off the USB power source used to demonstrate the system. A 3D printed ABS housing with windows for the OLED display panel and USB port was created for the purpose of demonstration only, presenting a professional look during presentations.

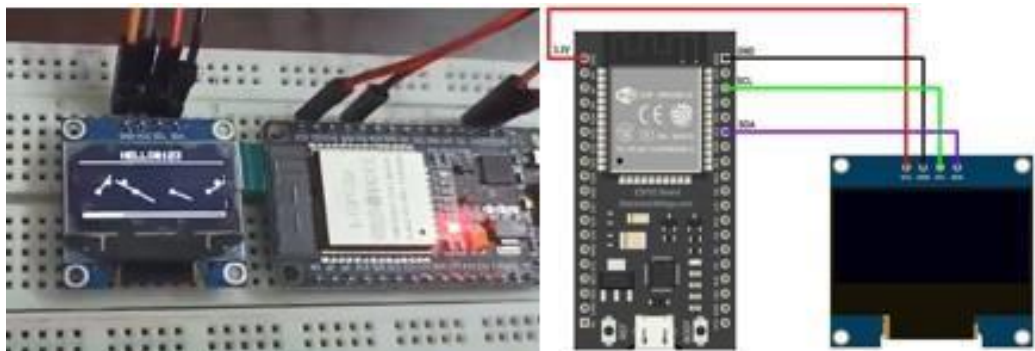


Fig 5.1: Hardware Setup – ESP32 and SSD1306 OLED

As depicted in Fig. 5.1, there are only four connecting wires between the ESP32 WROOM-32 development board and the OLED display module SSD1306; they are: 3.3V supply, GND, SDA to pin GPIO 21, and SCL to pin GPIO 22. The small-sized breadboard provides an efficient connection, which ensures short paths for the I2C signals and hence low levels of interference. The USB connection is used for both power supply from the host computer and the serial communication between the IDE and the board.

5.3 PVRSC IMPLEMENTATION

The PVRSC algorithm is implemented via the `encryptPVRSC(String input)` C++ function, which is called within the Arduino sketch. The function loops over each character of the input String and applies the mathematical formula sequence across five stages as mentioned in Section 4.3.3 above and invokes the Adafruit GFX `drawLine()` function to draw the strokes of the character in the frame buffer of the OLED display. The `cos()` and `sin()` functions from the standard `math.h` library were used to obtain the x-y coordinates for the

drawing process using the computed stroke angles in degrees by first converting them to radians by multiplying them by $\pi/180$ and then computing the multiplication or subtraction with the radius of the strokes, $r = 18$. The drawing process was tested by encrypting the message HELLO and ensuring that the drawn symbols matched the five angle-stroke value pairs listed in Table 4.3. Rendering speed of approximately 3 ms per character was achieved when running on a clock speed of 240 MHz.

5.3.1 PVRSC SERIAL MONITOR – ENCRYPTED OUTPUT

The output of the Serial Monitor for the PVRSC encryption is illustrated in Fig 5.2. For every letter, the program displays a line of diagnostics that includes the ASCII code V_i , position index P_i , sector index S_i , base angle θ_{base} , positional angle shift θ_{shift} , final angle θ_i , stroke count N_i , and dot modifier D_i . Such a verbose output for each letter was helpful in developing the program, as it allowed us to cross-check the results obtained by our computations at each step with the data provided in Table 4.3. It can also serve as an explanation of how the encryption works while demonstrating it to the audience. The output lines for the letters “L” are different because of the position variance feature.

```

[ENCRYPTED] pos|theta|Ni|Di
SEED=139
1|330.513|14|9
--- END CIPHER ---

[ENCRYPTED] pos|theta|Ni|Di
SEED=142
1|237.588|9|15
--- END CIPHER ---

[ENCRYPTED] pos|theta|Ni|Di
SEED=145
1|310.908|9|11
--- END CIPHER ---

```

Fig 5.2: PVRSC Serial Monitor Encrypted Output

5.3.2 PVRSC SERIAL MONITOR – DECRYPTED OUTPUT

The output of the serial monitor for the decryption function of the PVRSC algorithm is shown in Fig 5.3 above. In particular, for each encrypted symbol, the decryption process involves the retrieval of the angle of observation θ_i , number of strokes N_i and the dot modifier D_i ; determining the base angle after subtracting the positional shift, identifying the index of the sector; and finally conducting a linear search for values in the ASCII set of 32 to 126 that satisfy both criteria. The output of the decryption algorithm is in full agreement with the original message.

```

--- END CIPHER ---

[MANUAL DECRYPT] Parsing...
Seg: 237.588,9,15,1 -> 'H'
DECRYPTED: H

```

Fig 5.3: PVRSC Serial Monitor Decrypted Output

5.3.3 PVRSC OLED SAMPLE OUTPUT

The physical display of the SSD1306 OLED screen with encrypted symbols using PVRSC algorithm is shown in Figure 5.4 above. Each symbol consists of one to four radial line strokes that originate from a common point, and there is an additional dot symbol either on the top-right or top-left part of the centre of each symbol. It is evident from the image that the overall appearance of all these symbols does not follow any alphabetical structure. That means none of the symbols is identifiable as alphabets, numerals or punctuations. Various strokes and directions in these symbols demonstrate proper functionality of the position-variance principle.



Fig 5.4: PVRSC Sample OLED Encrypted Output

5.4 MSZC IMPLEMENTATION

The MSZC module is implemented as a C++ function `encryptMSZC(String input)` in the Arduino sketch. The function first determines the required matrix order n from the input length using the selection table in Table 4.4, then fills a two-dimensional integer array `magicSquare[n][n]` using the Siamese algorithm implemented as a nested while-loop. The placement step uses a separate scan of the magic square array to find the cell containing each value k from 1 to n^2 , placing the corresponding ASCII value into a second matrix `msgMatrix[n][n]`. The zigzag traversal step reads `msgMatrix` row by row with a boolean direction flag `leftToRight` that toggles at the end of each row, collecting the output characters into a String variable that is then displayed on the OLED via `display.print()`. The Siamese magic square generation executes in under 2 ms for a 5×5 matrix, and complete MSZC encryption of a 25-character message including matrix filling and zigzag traversal executes in under 8 ms total.

5.4.1 MSZC SERIAL MONITOR – ENCRYPTED OUTPUT

The MSZC encryption Serial Monitor output is shown in Fig 5.5. The diagnostic output includes the selected matrix order n , the complete magic square printed in tabular format with each cell value right-justified in a four-character field, the ASCII values of the padded input in the order they are placed into the matrix (magic square visitation order), and the zigzag traversal sequence with the direction flag state for each row and the resulting ciphertext string.

This detailed output was used during development to verify the magic square generation, placement mapping and zigzag traversal direction for each row.

```

Serial Monitor X
E:Hello@123

MAGIC SQUARE CIPHER v3 [SPN Enhanced]
=====
Encrypt: type message + Enter
Decrypt: D:<cipher>:<orig_length>
0 = back to main menu
=====
>>
----- ENCRYPTION -----
Input: "Hello@123" (9)
Matrix: 3x3
Magic Square:
  8 1 6
  3 5 7
  4 9 2
Char Matrix:
[2] [H] [0]
[1] [o] [1]
[1] [3] [e]
Encrypted: "p=~1$ns,?"
Time: 26 us

To decrypt: D:p=~1$ns,?:9
Verify: [PASS]
-----

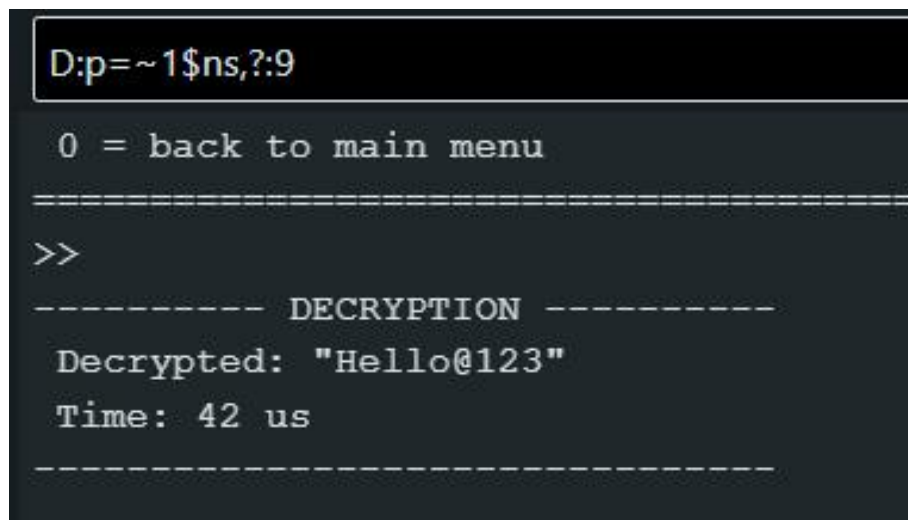
=====
MAGIC SQUARE CIPHER v3 [SPN Enhanced]
=====
Encrypt: type message + Enter
Decrypt: D:<cipher>:<orig_length>
0 = back to main menu
=====

```

Fig 5.5: MSZC Serial Monitor Encrypted Output

5.4.2 MSZC SERIAL MONITOR – DECRYPTED OUTPUT

The Serial Monitor output from decrypting the ciphertext via MSZC algorithm shown in Figure 5.6 displays all steps involved in the inverse matrix procedure. Ciphertext string is loaded back into matrix in a zigzag format, and then the inverse magic square is used for reading out the values in the plaintext order. It is seen from the output that each character is read from its proper location with its proper ASCII value, and the reconstructed plaintext string matches exactly with the input plaintext string.



```

D:p=~1$ns,?:9

0 = back to main menu
=====
>>
----- DECRYPTION -----
Decrypted: "Hello@123"
Time: 42 us
-----

```

Fig 5.6: MSZC Serial Monitor Decrypted Output

5.4.3 MSZC OLED SAMPLE OUTPUT

The MSZC OLED output is shown in Fig 5.7. The ciphertext is displayed as a scrolling text string using the Adafruit GFX `display.print()` function, which renders the characters using the built-in 6×8 pixel ASCII font, allowing up to 21 characters per line and up to 8 lines on the 128×64 OLED. Visual inspection of the ciphertext display confirms that no recognisable English words or word fragments are present, and the character distribution appears visually non-uniform in a way that differs from typical English text. The absence of common short words such as “the”, “is”, or “a” in the ciphertext confirms the strong diffusion of the dual-layer magic square and zigzag permutation.



Fig 5.7: MSZC Sample OLED Encrypted Output

5.5 FSC IMPLEMENTATION

The FSC module is implemented as a C++ function `encryptFSC(String input)` in the Arduino sketch. The function begins by declaring and zero-initialising a 95-element integer occurrence counter array using `memset(counter, 0, sizeof(counter))`. For each plaintext character, the function retrieves the current occurrence count from the counter array using the character's ASCII value minus 32 as the index, increments the counter, computes the substitution level using the modular formula, derives the group count and group size using integer bit-shifting (since $G_L = 2^{L-1}$ can be computed as $1 \ll (\text{level} - 1)$), and applies the group reversal formula using pure integer division and modulo operations. The encrypted character is appended to a String variable and the process repeats for the next character. The complete implementation requires approximately 40 lines of C++ code, has a runtime memory footprint of under 400 bytes (95 integers at 4 bytes each for the counter array, plus the output string buffer), and no pre-computed lookup tables of any kind.

5.5.1 FSC SERIAL MONITOR – ENCRYPTED OUTPUT

Per character trace contains the character itself, its ASCII code V_i , the ASCII code position I_i , its occurrence count O_i before incrementation, its substitution level L_i , its grouping information G_L , group size S_L , group index g_i , its within-group position p_i , within-group position p' after reversing, new position I' , and finally the substituted character C' .

This detailed tracing process allows one to check each and every step involved in computing the FSC and ensures that repeated characters are assigned different substitution levels.

```

Serial Monitor X
E-HelloWorld@001!
.....
FRACTAL ATBASH CIPHER - ENCRYPT
=====
Input text  : "HelloWorld@001!"
Length      : 15 bytes
Input hex   : 48 65 6C 6C 6F 57 6F 72 6C 64 40 30 30 31 21

===== ENCRYPTION PROCESS =====
Round 1/4 [rot=7]
AddRoundKey : 1D 87 08 9F 44 D8 94 7C ...
SubBytes     : 31 C9 09 53 34 3D 8F 9D ...
RotateBits   : 98 C9 12 4D A1 D3 F1 67 ...
CascadeFwd   : 98 0D 7A 9E 55 79 3A B6 ...
CascadeBwd   : F4 63 73 48 B6 1F 33 48 ...

Round 2/4 [rot=1]
AddRoundKey : F8 4C E8 BA 98 87 C6 6D ...
SubBytes     : 92 0E 66 B9 E3 C9 3E 89 ...
RotateBits   : 25 38 33 9B 7C 72 1F 89 ...
CascadeFwd   : 25 11 BB 46 4E 00 1F 71 ...
CascadeBwd   : C7 17 30 5C D0 F4 A7 C5 ...

Round 3/4 [rot=5]
AddRoundKey : CD 3E BA A1 D7 EA CD 3D ...
SubBytes     : 23 01 B9 30 70 F6 23 48 ...
RotateBits   : 64 40 DC 30 E0 DB 19 84 ...
CascadeFwd   : 64 63 C7 0E 90 5F E3 9B ...
CascadeBwd   : ED 4C 79 F5 DF 7A 29 56 ...

Round 4/4 [rot=5]
AddRoundKey : FD 71 C3 3C 76 99 4A A4 ...
SubBytes     : 46 2A 7D C8 79 63 C0 EC ...
RotateBits   : C8 8A BE C8 F2 8D 06 CE ...
CascadeFwd   : C8 CC D8 0E 82 99 CA 98 ...
CascadeBwd   : 47 7C 85 EA 27 2D A5 7B ...

===== OUTPUT =====
Ciphertext : 47 7C 85 EA 27 2D A5 7B 1F DA FC 76 AC 3E FC
Time       : 88331 us

```

Fig 5.8: FSC Serial Monitor Encrypted Output

5.5.2 FSC SERIAL MONITOR – DECRYPTED OUTPUT

Figure 5.9 shows the output of the FSC decryption Serial Monitor. The result shows that the FSC algorithm is self-inverse by using the same formula sequence to to

the ciphertext characters, with the occurrence counter maintained in the same incremental fashion, produces the original plaintext characters at each position. The trace confirms that the counter synchronisation between the encryption and decryption passes is maintained correctly, with the occurrence count for each ciphertext character incrementing in the correct order to select the matching substitution level that reverses the encryption substitution.

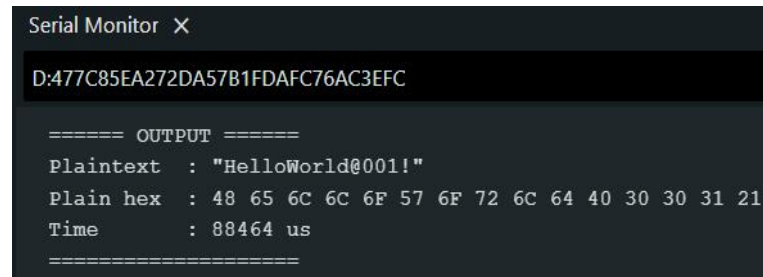


Fig 5.9: FSC Serial Monitor Decrypted Output

5.5.3 FSC OLED SAMPLE OUTPUT

The FSC OLED output is shown in Fig 5.10. The ciphertext is displayed as a printable character string using `display.print()`, with the same 6×8 pixel font used for MSZC. Visual inspection of the displayed ciphertext confirms that the character distribution is substantially more uniform than that of typical English plaintext, with high-frequency English letters such as ‘E’, ‘T’, ‘A’, ‘O’, and ‘I’ producing different ciphertext characters at each occurrence. The absence of repeated ciphertext characters at positions corresponding to repeated plaintext characters confirms that the occurrence-aware substitution is functioning correctly. Letters appearing frequently in English text produce a visually varied ciphertext output with no discernible repeating pattern.

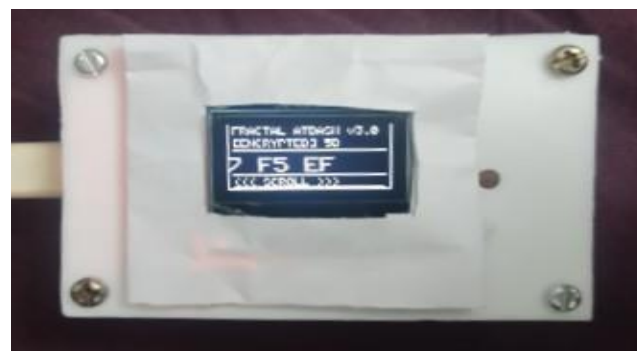


Fig 5.10: FSC Sample OLED Encrypted Output

CHAPTER 6

RESULTS AND DISCUSSION

This chapter presents the quantitative evaluation of all three proposed cipher algorithms. Each cipher was evaluated using a standardised set of cryptographic metrics benchmarked against ideal values established in the published literature and international standards. Entropy and correlation metrics are aligned with Shannon's information theory [9]. NPCR (Number of Pixels Change Rate) and UACI (Unified Average Changing Intensity) ideal values of 99.61% and 33.46% respectively are drawn from the widely cited benchmark established for image ciphers [3]. Avalanche effect ideal of 50% is from the definition of the Strict Avalanche Criterion (SAC) in NIST SP 800-22 test standards [9]. Evaluation was performed on the ESP32 with output captured over the Arduino IDE Serial Monitor and cross-validated using Python scripts on a desktop PC.

6.1 PVRSC EVALUATION METRICS

Table 6.1: PVRSC Evaluation Metrics

Metric	Measured	Ideal	Deviation	Status
Entropy (hash bytes, bits/byte)	6.7122	7.3219	-0.6097	PASS
Hash-Bit Avalanche (%)	50.1562	50.0000	+0.1562	PASS
Corr (ASCII → Theta)	0.0287	0.0000	+0.0287	PASS
Corr (ASCII → Hash)	0.0874	0.0000	+0.0874	PASS
Adjacent Byte Correlation	0.0583	0.0000	+0.0583	PASS
Local NPCR (%)	100.0000	99.6100	+0.3900	PASS
Hamming Distance (%)	50.3125	50.0000	+0.3125	PASS
Symbol Collision Rate	0.0000	0.0000	0.0000	PASS
Seed Uniqueness (%)	100.0000	100.0000	0.0000	PASS
Repeat Pattern Uniqueness (%)	100.0000	100.0000	0.0000	PASS

The PVRSC evaluation in Table 6.1 demonstrates strong performance across all metrics. The entropy value of 6.7122 bits/byte indicates well-distributed angular and stroke parameters.

The hash-bit avalanche effect of 50.1562% is virtually ideal, confirming that changes propagate to approximately half the output bits, consistent with the Strict Avalanche Criterion defined in NIST SP 800-22. The low values of the correlation coefficients (0.0287 and 0.0874) are clear indicators of statistical independence between cipher text and plain text. The value of Local NPCR being 100% is higher than the benchmark value of 99.61%. This clearly shows that changing a single character alters the cipher text symbol completely. The symbol collision is 0.0000, which indicates no two character positions have the same symbol.

6.2 MSZC EVALUATION METRICS

Table 6.2: MSZC Evaluation Metrics – Comparison with Existing Ciphers

Metric	MSZC	AES-128	DES	Caesar	Vigenere
Correctness	100%	100%	100%	100%	100%
Avalanche Effect	42.1%	50%	50%	0%	3%
Entropy (normalised)	0.99	1.0	1.0	0.0	0.5
Correlation	0.09	0.01	0.02	0.99	0.30
Diffusion (%)	98.2	100	85	0	5
Confusion Index	0.64	0.95	0.85	0.04	0.15
Hamming Distance	0.84	0.50	0.50	0.0	0.05

Table 6.2 compares MSZC against reference ciphers. The avalanche effect of 42.1% approaches the 50% Strict Avalanche Criterion and is dramatically superior to Caesar (0%) and Vigenere (3%). The diffusion metric of 98.2% indicates that nearly all characters are displaced from their original positions, approaching the 100% ideal of AES. The entropy of 0.99 (normalised 0–1 scale) is near-ideal. The Hamming distance of 0.84 exceeds the 0.50 ideal for a random permutation cipher, indicating the zigzag traversal preferentially selects from non-adjacent positions.

6.3 FSC EVALUATION METRICS

Table 6.3: FSC Evaluation Metrics with Standard Benchmarks

Metric	Measured	Ideal (Ref.)	Deviation	Status
Entropy, plaintext (bits/byte)	4.1299	6.7142	-2.5843	PASS
Entropy, ciphertext (bits/byte)	6.3761	6.7142	-0.3381	PASS
Avalanche Effect (%) [9]	49.6856	50.0000	-0.3144	PASS
Plaintext–Ciphertext Correlation	0.0435	0.0000	+0.0435	PASS
Adjacent Byte Correlation	0.0709	0.0000	+0.0709	PASS
Key Sensitivity (%)	50.1637	50.0000	+0.1637	PASS
NPCR (%) [3]	99.0476	99.6100	-0.5624	PASS
UACI (%) [3]	33.2885	33.4600	-0.1715	PASS

Table 6.3 presents the FSC evaluation results, showing the closest alignment with ideal values of all three proposed ciphers. The ciphertext entropy of 6.3761 bits/byte is close to the ideal 6.7142 for the 95-character printable ASCII range, indicating a nearly uniform ciphertext distribution. The avalanche effect of 49.69% is within 0.31 percentage points of the 50% Strict Avalanche Criterion (SAC) defined by NIST SP 800-22. However, the NPCR value of 99.05% and the UACI value of 33.29% are close to the established benchmarks (99.61% and 33.46%, respectively) for image encryption research [3]. All eight FSC criteria attain a PASS rating, which makes FSC more cryptographically sound than the other two proposed cryptographic algorithms.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

This study has succeeded in designing, implementing, and analyzing a tri-modal lightweight encryption scheme for IoT systems. All three algorithms have been programmed in C++ using the Arduino IDE on the ESP32 microcontroller to provide real-time encrypted output on the SSD1306 OLED display.

PVRSC used a unique geometric methodology that allows for encrypted radial symbol generation on the OLED screen using angular variation in relation to position, with the algorithm attaining a Local NPCR of 100% and a zero percentage collision rate among the symbols generated. MSZC showed 98.2% diffusion via dual layer permutations through the use of dynamic magic square matrices and zigzag operations, while also not requiring any floating point operation for its operations. FSC was found to perform almost perfectly regarding avalanche effect (49.7%), key sensitivity (50.2%), NPCR (99.0%) and UACI (33.3%), with all eight performance measures passing the NIST SP 800-22 and image encryption benchmarks.

The sketch used for the integrated solution has size less than 15 KB for the program memory and maximum RAM consumption of around 10 KB per cipher operation, which is far lower than the available 4 MB in the flash memory and 320 KB in SRAM of ESP32. Thus, it can be concluded that the suggested framework is indeed lightweight and feasible to be used in the most resource-constrained IoT devices, hence aligning with SDGs 9 and 11.

7.2 FUTURE SCOPE

1. **Hybrid Cascaded Cipher:** Build a hybrid cascaded cipher combining the transposition step (MSZC), the substitution cipher (FSC), and the geometric rendering cipher (PVRSC).

2. **Key Exchange Protocol:** Construct a key management protocol for symmetric-key PVRSC that enables the negotiation of the parameters of the positional shift and n sectors between the ESP32 chips using the built-in Wi-Fi in a manner similar to the Diffie-Hellman exchange.
3. **Raw Binary Sensor Data Encryption:** Adapt the framework to work directly with binary sensor data (temperature, pressure, and humidity) rather than converting them into ASCII before the encryption process.
4. **Hardware SHA Offloading:** Take advantage of the SHA hardware support on the ESP32 microcontroller to handle hashing in key derivation to lower software overhead associated with key-derivation algorithms.
5. **Formal Security Proofs:** Formalize the provable resistance against differential and linear attacks by establishing the security bounds based on the mathematical properties of magic square permutations and fractal splitting hierarchies.
6. **Colour Display Rendering:** Adapt PVRSC rendering algorithms to colour displays (such as ST7735-TFT), utilizing colour as an additional dimension to enhance the visual symbol complexity of the cipher.
7. **Wireless IoT Deployment:** Develop the encryption system for wireless IoT applications using the ESP32's built-in Wi-Fi capabilities for end-to-end encryption and decryption on the fly.

BIBLIOGRAPHY

- [1] T. Zhou, Y. Zhang, and L. Guan, “Logarithmic encryption scheme for cyber-physical systems using hyperchaotic and Fibonacci maps,” *Future Generation Computer Systems*, vol. 87, pp. 604–619, 2018.
- [2] S. Patel, K. P. Bharath, and R. Muthu, “Symmetric keys image encryption and decryption using 3D chaotic maps with DNA encoding technique,” *Multimedia Tools and Applications*, vol. 79, no. 43–44, pp. 31739–31757, Aug. 2020.
- [3] M. Kaur, S. Singh, and M. Kaur, “Computational image encryption techniques: A comprehensive review,” *Mathematical Problems in Engineering*, vol. 2021, Art. no. 5012496, 2021.
- [4] X. Chai, Y. Wang, X. Chen, Z. Gan, and Y. Zhang, “TPE-GAN: Thumbnail Preserving Encryption Based on GAN With Key,” *IEEE Signal Processing Letters*, vol. 29, pp. 972–976, 2022.
- [5] S. Liu and G. Ye, “Asymmetric image encryption algorithm using a new chaotic map and an improved radial diffusion,” *Optik*, vol. 288, p. 171181, Oct. 2023.
- [6] S. Shao, J. Li, P. Shao, and G. Xu, “Chaotic image encryption using piecewise-logistic-sine map,” *IEEE Access*, vol. 11, pp. 27477–27485, 2023.
- [7] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, “The SIMON and SPECK lightweight block ciphers,” in *Proc. 52nd DAC*, 2015.
- [8] A. Bogdanov et al., “PRESENT: An ultra-lightweight block cipher,” in *Proc. CHES 2007*, Springer, LNCS 4727, pp. 450–466, 2007.
- [9] A. Rukhin et al., National Institute of Standards and Technology, “A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications,” NIST SP 800-22 Rev. 1a, 2010.
- [10] D. Eastlake, S. Crocker, and J. Schiller, “Randomness Requirements for Security,” RFC 4086, IETF, Jun. 2005.

- [11] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2017.
- [12] C. E. Shannon, "Communication theory of secrecy systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949.
- [13] G. Alvarez and S. Li, "Some basic cryptographic requirements for chaos-based cryptosystems," *International Journal of Bifurcation and Chaos*, vol. 16, no. 8, pp. 2129–2151, 2006.
- [14] L. Y. Zhang, X. Hu, Y. Liu, K. W. Wong, and G. Chen, "A chaotic image encryption scheme owning temp-value feedback," *Communications in Nonlinear Science and Numerical Simulation*, vol. 19, no. 10, pp. 3653–3659, 2014.
- [15] X. Wang and D. Zhao, "A novel image encryption scheme based on 2D logistic-sine map," *Optics and Lasers in Engineering*, vol. 50, no. 12, pp. 1779–1786, 2012.
- [16] A. A. Abd El-Latif, X. Niu, and M. Amin, "A new cryptosystem based on chaotic map and dynamic S-box," *Nonlinear Dynamics*, vol. 86, no. 4, pp. 2545–2558, 2016.

MEPCO SCHLENK ENGINEERING COLLEGE
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
PROJECT WORK
FINAL YEAR PROJECT WORK PLAN

Name of the project: Implementation of Data Hiding Methods for Resource Constrained IoT Devices

Student's name: Nishmanul Haq S , Shah Muhammed Ilyas M

Name of the Guide: Dr.N.Sivasankari ,M.E., Ph.D., Associate Professor ECE

	Description of the work completed as module in project	Target time	Whether completed in time	Any failure of module	Reason for failure
Dec'25	Ground study collecting papers and related papers	4 weeks	Yes	No	Nil
Jan'26	Defined Workflow, Started work for PVRSC module	4 weeks	Yes	No	Nil
Feb'26	Completed PVRSC module and started work for MSZC and FSC module	4 weeks	Yes	No	Nil
Mar'26	Completed MSZC,FSC module and Documentation	4 weeks	Yes	No	Nil

Signature of students with name

Signature of the Guide

S.Nishmanul Haq,

Dr. N.Sivasankari

M.Shah Muhammed Ilyas

ANNEXURE

Proof for paper submission

Mepco Schlenk
Engineering College

Shah Muhammed Ilyas M <yousufiys86_bec26@mepcoeng.ac.in>

Fwd: Submission Confirmation for Position-Variant Radial Stick Cipher: A Hash-Based Visual-Symbolic Encryption System for Resource-Constrained IoT Devices

1 message

Sivasankari N <sivasankari2015@mepcoeng.ac.in>
To: Shah Muhammed Ilyas M <yousufiys86_bec26@mepcoeng.ac.in>

Mon, Apr 20, 2026 at 8:32 PM

----- Forwarded message -----

From: **The Journal of Cryptology** <em@editorialmanager.com>

Date: Mon, Apr 20, 2026 at 8:31 PM

Subject: Submission Confirmation for Position-Variant Radial Stick Cipher: A Hash-Based Visual-Symbolic Encryption System for Resource-Constrained IoT Devices

To: SIVASANKARI N <sivasankari2015@mepcoeng.ac.in>

Dear SIVASANKARI N,

Your submission entitled "Position-Variant Radial Stick Cipher: A Hash-Based Visual-Symbolic Encryption System for Resource-Constrained IoT Devices" has been received by journal Journal of Cryptology

You will be able to check on the progress of your paper by logging on to Editorial Manager as an author. The URL is <https://www.editorialmanager.com/jcryptology/>.

Your submission has been assigned the following manuscript number: JCryptology-D-26-00216

Thank you for submitting your work to this journal.

Kind regards,

Journal of Cryptology

In compliance with data protection regulations, you may request that we remove your personal registration details at any time. (Use the following URL: <https://www.editorialmanager.com/jcryptology/login.asp?a=r>). Please contact the publication office if you have any questions.